



Article

A Multimodal Phishing Website Detection System Using Explainable Artificial Intelligence Technologies

Alexey Vulfin ^{1,*}, Alexey Sulavko ^{2,*}, Vladimir Vasiliev ¹, Alexander Minko ¹, Anastasia Kirillova ¹
and Alexander Samotuga ^{2,*}

¹ Department of Computer Science and Information Security, Ufa University of Science and Technology, Zaki Validi Str, h. 32, Ufa 450076, Russia; vasilev.vi@ugatu.su (V.V.); minko.av@uust.ru (A.M.); kirillova.ad@ugatu.su (A.K.)

² Department of Comprehensive Information Protection, Omsk State Technical University, Mira, h. 11, Omsk 644050, Russia

* Correspondence: vulfin.am@ugatu.su (A.V.); sulavich@mail.ru (A.S.); samotugasashok@mail.ru (A.S.); Tel.: +79-5339-490-54 (Alexey Sulavko)

Abstract

The purpose of the present study is to improve the efficiency of phishing web resource detection through multimodal analysis and using methods of explainable artificial intelligence. We propose a late fusion architecture in which independent specialized models process four modalities and are combined using weighted voting. The first branch uses CatBoost for URL features and metadata; the second uses CNN1D for symbolic-level URL representation; the third uses a Transformer based on a pretrained CodeBERT for the homepage HTML code; and the fourth uses EfficientNet-B7 for page screenshot analysis. SHAP, Grad-CAM, and attention matrices are used to interpret decisions; a local LLM generates a consolidated textual explanation. A prototype system based on a microservice architecture, integrated with the SOC, has been developed. This integration enables streaming processing and reproducible validation. Computational experiments using our own updated dataset and the public MTLP dataset show high performance: F1-scores of up to 0.989 on our own dataset and 0.953 on MTLP; multimodal fusion consistently outperforms single-modal baseline models. The practical significance of this approach for zero-day detection and false positive reduction, through feature alignment across modalities and explainability, is demonstrated. All limitations and operational aspects (data drift, adversarial robustness, LLM latency) of the proposed prototype are presented. We also outline areas for further research.

Keywords: phishing; multimodal detection; late fusion; explainable AI; SOC; cybersecurity



Academic Editor: Edgar Weippl

Received: 19 November 2025

Revised: 19 December 2025

Accepted: 29 December 2025

Published: 4 January 2026

Copyright: © 2026 by the authors.

Licensee MDPI, Basel, Switzerland.

This article is an open access article distributed under the terms and conditions of the [Creative Commons Attribution \(CC BY\)](https://creativecommons.org/licenses/by/4.0/) license.

1. Introduction

In today's digital world, attacks on information systems using social engineering are becoming increasingly common. These attacks aim to deceive users into revealing confidential information, which the attacker then uses for personal enrichment and to harm the owner of the information resource. According to the Russian company Positive Technologies [1], the number of computer attacks in Russia increased by 26% in 2024. A substantial proportion of computer attacks, affecting both organizations (50%) and individuals (88%), are attributed to social engineering. The most common social engineering attack is phishing—an attack orchestrated by an attacker to force a user to interact with an infected (phishing) website in order to extract critical credentials (logins, passwords, payment details) and inject malware into the system. According to the international

working group APWG (Anti-Phishing Working Group), the number of phishing attacks against online payments and the financial (banking) sector increased significantly in the first quarter of 2025, amounting to 30.9% of the total number of attacks [2].

There are three main methods of organizing phishing attacks [3]:

- Malicious links, which appear as impostor websites infected with malware;
- Malicious file attachments, which are infected with malware to compromise the user's computer or files;
- Fraudulent data entry forms, which prompt the user to enter their login credentials or other sensitive information to log in.

Unlike the past, when attackers required advanced technical skills and knowledge of web programming, scripting, social engineering techniques, and so on, a dedicated industry (criminal environment) called Phishing-as-a-Service (PaaS) has emerged in recent years, opening up the organization and execution of successful, highly organized phishing attacks to a wide audience of non-professional criminals. This situation is driving the rapid growth of the phishing threat and highlights the urgent need to take significant measures to neutralize it.

As noted in some studies [4–7], existing traditional methods of detecting phishing web resources, such as the use of blacklists of malicious sites and analysis of uniform resource locators (URLs)—links to specific web pages or the addresses of other web resources on the internet—cannot accurately identify phishing sites, resulting in a significant number of false positives.

In this context, the use of machine learning (ML) methods, such as artificial neural networks (NNs), support vector machines (SVMs), random forest (RF) algorithms, and others, is promising. These methods allow for the analysis and classification of websites based on a set of characteristic URLs associated with phishing. The solution to the problem of detecting phishing websites is reduced to the problem of the binary classification of URLs in a vector feature space: determining whether a URL belongs to one of two classes—legitimate or malicious (phishing). The advantages of using machine learning methods are higher accuracy in the presence of noisy and incomplete input data, the ability to adapt in real time and retrain when the training data change, and the ability to detect new types of phishing attacks. Thus, they ensure greater efficiency in detecting phishing websites.

Additional advantages in solving the problem of detecting phishing websites are revealed by the emergence of modern advanced artificial intelligence (AI) technologies based on the application of deep learning methods (DL), including convolutional neural networks (CNNs), long short-term memory networks (LSTM), Transformers, large language models (LLMs) [8–10], and hybrid (multimodal) AI systems [11–13]. A distinctive feature of this class of AI methods and systems is the use of additional information about the web resource as source code and a visual image (image) of the web page besides a set of symbolic URL features; this increases the dimensionality of the feature space under study, attracting additional expert knowledge about the structure of phishing websites at the classification stage, which ultimately increases the reliability of the decisions made. Such models can be classified as multimodal (modality refers to the type of input/output data and methods of extracting features from these data for subsequent processing in a machine learning model [14]).

As noted in some studies, the approaches discussed above (machine and deep learning methods, multimodal AI systems) have one major drawback: they are implemented based on a “black box” architecture, i.e., they are not transparent to the receiver of the results (in this case, an information security (IS) specialist) and have low interpretability (there is a result, but it is unclear why this result was obtained or what evidence supports it). This explains the increased interest in creating “explainable” AI (XAI) and, in particular, in cre-

ating a new generation of phishing website detection systems based on it, which not only have high accuracy in detecting infected sites but also high interpretability (explainability) regarding the obtained results (examples illustrating the construction of such systems can be found in [15–17]).

It should also be noted that building phishing website detection systems based on the previously mentioned ML, DL, and XAI methods in practice is challenged by a number of open questions, such as

- The ambiguity in choosing a suitable dataset (training dataset);
- The composition of features included in the dataset;
- The architecture of the classifier model;
- The values of its hyperparameters;
- The means of selecting effective training algorithms (tuning) for the detection system;
- The mechanisms in interpreting the results, considering the natural limitations associated with the required computational costs of system implementation.

Thus, the present study is aimed at developing and experimentally evaluating an industrial-grade multimodal phishing web resource detection system designed to

- Incorporate specialized models for four heterogeneous modalities (URL features and metadata, URL symbolic representation, HTML code, and web page screenshot) within a late fusion architecture;
- Incorporate a comprehensive, end-to-end explainable AI pipeline (SHAP, Grad-CAM, attention matrix analysis) that subsequently generates human-readable explanations utilizing a local LLM;
- Guarantee operational suitability in an SOC environment by means of microservice implementation, TI/SIEM integration, and testing against zero-day links and obfuscated attacks.

The scientific novelty of this work consists of the following aspects:

1. A multimodal, late fusion architecture for the detection of phishing web resources is proposed. In this architecture, independent, specialized models for four modalities (CatBoost for URL features and metadata, CNN1D for symbolic URLs, a CodeBERT Transformer for HTML code, and EfficientNet-B7 for screenshots) are combined via a trainable meta-classifier that optimises the F1-score for the “phishing” class.
2. A comprehensive, end-to-end explainable AI (XAI) pipeline is developed. It includes SHAP coefficients for tabular features, Grad-CAM and integrated gradients for symbolic and visual models, Transformer attention matrix analysis for HTML code, and the subsequent aggregation of these explanations by a local LLM into structured text reports intended for SOC analysts.
3. An industrial-grade prototype system based on a microservice architecture with integration into the SOC/TI/SIEM framework is implemented and experimentally tested. It can process 0.5–1.1 thousand events per second, degrading to a subset of models while maintaining acceptable detection quality.
4. A quality assessment is performed, which includes
 - A comparison of single-modal and multimodal configurations on a proprietary, updated dataset (3.2 million URLs) and the public MTLTP dataset, where the multimodal ensemble achieves an F1-score of 0.989 and 0.953, respectively;
 - An exploratory assessment of resistance to typical obfuscation techniques;
 - An expert review of the explanation utility with SOC analysts, using Fleiss’s κ and Cronbach’s α to demonstrate the XAI component’s high consistency and practical applicability.

This article is structured as follows. Section 2 provides a comprehensive analysis and the systematization of relevant studies on the development of a machine and deep learning-based phishing detection system using explainable artificial intelligence mechanisms. Section 3 presents the proposed approach to building a multimodal phishing detection system based on deep learning and explainable artificial intelligence and discusses the system's architecture, the composition of its software modules, and the tasks that it performs. Section 4 presents the results of comparative experiments that confirm the viability and effectiveness of the developed system and offers recommendations for its practical application. The Conclusions provide insights into the potential and prospective uses of multimodal systems.

2. Analysis of Relevant Works

2.1. Description of Data Collection Methodology for Review of Sources

The objective of the review was to systematize and analyze modern approaches to developing multimodal phishing detection systems using XAI methods for the period from 2020 to 2025. Peer-reviewed studies were identified and extracted through the following indexing databases: IEEE Xplore, ACM Digital Library, Springer, ScienceDirect, arXiv, Google Scholar, e-Library, e-Library.ru, and cyberlin.ru.

Key search queries used to find sources for the review were as follows:

- “multimodal phishing detection” AND (“explainable AI” OR “XAI”);
- “deep learning” AND “phishing detection” AND (“SHAP” OR “LIME”);
- “large language models” AND “phishing”;
- “visual phishing detection” AND “interpretability”;
- “multimodal fusion” AND “cybersecurity”.

Inclusion criteria: English or Russian peer-reviewed articles in scientific journals and conferences; technical reports from leading security organizations; open-source projects with scientific substantiation; studies with empirical results.

Exclusion criteria: publications without a quantitative assessment of performance; work not related to multimodal analysis or to XAI.

2.2. Review of Major Publications on Research into Multimodal Phishing Detection Systems

A large number of studies have examined the practical application of machine learning and deep learning methods in detecting phishing websites (see, for example, the comprehensive reviews [5,7,10,13]). Table 1 provides typical examples of the implementation of ML- and DL-based systems, listing the authors of the relevant publications, the datasets used, the ML and DL models, and the accuracy achieved in detecting phishing sites.

An analysis of the content of these publications (Table 1) leads to the following conclusions:

1. There are now fairly representative open datasets containing a large number of URL samples with a wide variety of characteristics for both phishing sites (Kaggle, Phish-Tank, OpenPhish, etc.) and legitimate sites (Alexa Rank, crawlcommon, etc.), which can be used in the training phase for phishing web detection systems (see also [18]).
2. An important stage preceding the training of such systems is a detailed study (pre-processing) of the set of features of phishing and legitimate websites contained in the dataset, including their normalization and coding, the analysis of correlations and informational significance, the possible reduction of the dimensionality of the feature space, etc., using methods of feature selection (detailed information about this can be found in [19–21]).
3. Based on their research, the authors of several publications [21–23] pay close attention to the comparative assessment of the effectiveness of different ML models (as used in

the specific issue of phishing web addresses) using such quality metrics as accuracy, recall, accuracy, and F1-score (harmonic mean). Based on their research, random forest algorithms, decision trees, and multilayer perceptrons are preferred.

4. A hybrid ML model based on a set of classifiers [11] (for example, implemented by an RF algorithm) is particularly interesting in terms of improving the accuracy of phishing detection, where each individual classifier interacts with its own set of URL features and the final decision on the presence or absence of a phishing threat is taken by an arbitrator (MLP) using a separate voting system.
5. The combination of the capabilities of natural language text analysis (natural language processing, NLP), including such features as URLs, text blocks, domains, and logs, with the processing of visual elements (screenshots) of a web page, as well as structural features of HTML code, within the framework of a multimodal concept [10,24,25], allows one not only to increase the precision and recall but also to predict the development of new types of phishing attacks, including generative adversarial attacks.

Table 1. The summary table shows the use of machine and deep learning methods in detecting phishing attacks.

No.	Authors	Dataset	ML/DL Models	Accuracy
1.	Al-Diabat M. [19]	University of Irvine Repository (11,000 URLs, based on PhishTank and Yahoo Directory, 30 features)	Classifiers based on C4.5 and IREP algorithms The scikit-learn library version 1.8 was used.	C4.5—96% IREP—95%
2.	Mahajan R., Siddavatam I. [22]	Phishtank (19,653 URLs) + Alexa Rank (17,058 URLs)	Decision trees (DT), RF, SVM The scikit-learn library version 1.8 was used.	RF—97.14%
3.	Almomani A., Alauthman M., Shatnawi M.T. et al. [20]	Huddersfield University Dataset 1 (2456 URLs, 30 features); Dataset 2 (10,000 URLs, 48 features)	16 ML models (RF, classification and regression trees (CART), logistic regression (LR), SVM, NN, Bayesian additive regression trees (BART), etc.) The scikit-learn library version 1.8 was used.	RF—96% (Dataset 1) and 98% (Dataset 2)
4.	Khera M., Prasad T., Xess L.D., et al. [23]	Phishing website dataset	9 ML models (RF, DT, LR, K-nearest neighbors (kNN), XGBoost, XBNNet, etc.) The scikit-learn library version 1.8 was used.	RF—97.3%
5.	Alazaidah R., Al-Shaikh A., Almousa M.R. et al. [21]	Dataset 1 (11,055 URLs, 30 features, 2 classes) Dataset 2 (1353 URLs, 9 features, 3 classes)	24 ML models, 6 learning strategies (Naive Bayes, LR, MLP, AdaBoost, RF, random tree, etc.) The scikit-learn library version 1.8 was used.	Filtered Classification J-48—90.76% (Dataset 1); RF—97.26% (Dataset 2)
6.	Aslam, S., Aslam, H., Manzoor, A. et al. [4]	Dataset 1 (DS1): Benign Yandex sites Dataset 2 (DS2): Benign sites from Common Crawl, the Alexa database, and phishing sites from PhishTank	LSTM	96.04%
7.	Ghaleb F.A., Alsaedi M., Saeed F., et al. [11]	Malicious URLs dataset (651,191 URLs)	Hybrid ML model: $3 \times$ RF + multilayer perceptron (MLP)	96.80%
8.	Krotov E.Yu., [10]	Phishing website dataset (10,000 URLs) + Common Crawle	Multimodal system: CNN + LSTM (image + text)	92%
9.	Lee J., Lim P., et al. [24]	Open Phish (1585 URLs) + Alexa Rank (3000 URLs)	Multimodal system: GPT-4-turbo, Claude 3, Gemini Pro-Vision 1.0	GPT-4—92% Claude 3—90%
10.	Syed Sh. A., [25]	PhishTank, APWG, Open Phish	BERT (text) + ResNet + VGG16 (image)	96.2%

According to the review, multimodal phishing detection systems can be categorized on the basis of three main criteria: the type of modality used, the strategy for combining them, and the level of explainability integration.

2.3. Classification of Data Types for Feature Generation in the Detection of Phishing Attacks

Figure 1 and Table 2 show the possible classification [13,24,26–30] of data types for generating features (modalities) in modern systems.

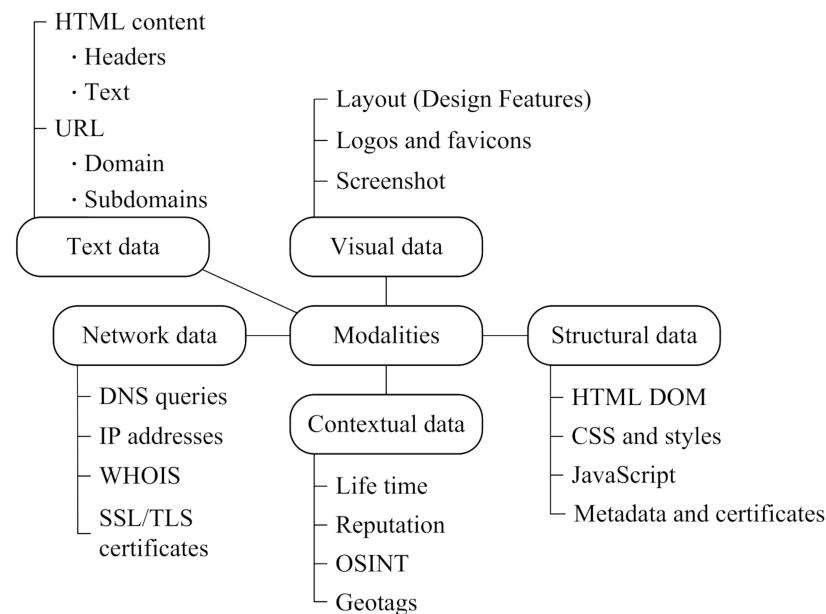


Figure 1. Classification of data types for feature generation in phishing attack analysis.

Table 2. Types of modalities in modern detection systems for phishing attacks.

Modality Type	Features	Extraction Methods
Text	URL, email body, HTML content, domain/subdomain	NLP, tokenization, embeddings (BERT, Word2Vec)
Visual	Page screenshots, logos, favicons, color schemes, layout	CNN, transfer learning (ResNet, VGG), object detection
Structural	HTML DOM, CSS, JavaScript, metadata, certificates	DOM parsing, graph analysis, feature extraction
Network	DNS queries, IP addresses, WHOIS, SSL certificates	Network analysis, OSINT, reputation services

The following itemization enumerates the features associated with various modalities [7,12,31–33].

- The main categories of URL features: lexical, syntactic, semantic, entropic, statistical;
- The main categories of structural HTML features: DOM structural, data entry form structure, characteristics of the page's script component, CSS and style features of the design template, and semantic HTML features;
- The main categories of visual features: logo detection, layout analysis, color analysis, visual similarity, and image quality.

Table 3 shows the comparative characteristics of the achieved accuracy for multimodal detection systems for phishing attacks.

An analysis of publications on the classification of features shows the following:

- URL features provide fast initial analysis with relatively high accuracy (up to 93%) but are vulnerable to attacks by URL obfuscation;
- HTML features provide the most detailed information about the attack structure and are particularly effective in detecting credential phishing;
- Visual features are critical in detecting brand impersonation and visual cloning attacks, achieving 94–98% accuracy, but require significant computational resources;
- Multimodal integration outperforms single-modal approaches in the F1-score by 8–15 percent, but it increases the time to classify (up to 5–10 s);

- Feature engineering remains an important step even in the use of deep learning, as hand-selected and hand-crafted features ensure interpretability and resilience to adversarial perturbations.

Table 3. Distribution of modalities in modern detection systems for phishing attacks.

Combination of Modalities	Number of Systems	Average Accuracy	Typical Architectures of ML Models
URL + HTML	18	96.5–99.2%	CNN + LSTM, BERT [34,35]
URL + Visual	12	93.4–97.8%	CNN + ResNet/VGG [36,37]
URL + HTML + Visual	8	98.0–99.4%	Transformers, LLM [27,28]
All 5 types	3	98.4–99.6%	Agent-based LLM systems [27]

There are four main feature fusion strategies across all modalities [28,34,38] (Table 4 and Figure 2).

1. Early fusion: Concatenates the properties of all modalities at the input level, followed by processing with one model. Advantages: Captures low-level correlations of features, simple architecture of the final model. Disadvantages: Loss of specificity of the modality, complexity of training.
2. Intermediate fusion: Each modality is processed by a separate encoder and then the features are combined at an intermediate level by means of an attentional mechanism. It provides a balance between specificity and modality interaction.
3. Late fusion: Independently train specialized models for each modality with decision aggregation by means of voting or stacking. Advantages: Model specialization, modularity, parallelism. Best results for heterogeneous data.
4. Hybrid fusion: Combination of early and late fusion with adaptive weights. This strategy shows maximum performance with high architectural complexity.

Table 4. Comparative analysis of modality fusion strategies.

Fusion Strategies	Advantages	Disadvantages	Examples of Systems
Early Fusion	– Accounting for low-level correlations – Simple architecture	– Loss of modality specificity – Training difficulty	MultiPhish [38]
Intermediate Fusion	– Balance of specificity and interaction – Mechanisms of attention	– Average learning difficulty	PhishTransformer [35]
Late Fusion	– Model specialization – Modularity – Parallelization	– Loss of intermodule connections	Yoon et al. [33]
Hybrid Fusion	– Maximum performance; adaptability	– High model complexity – Risk of overfitting	KnowPhish [28]

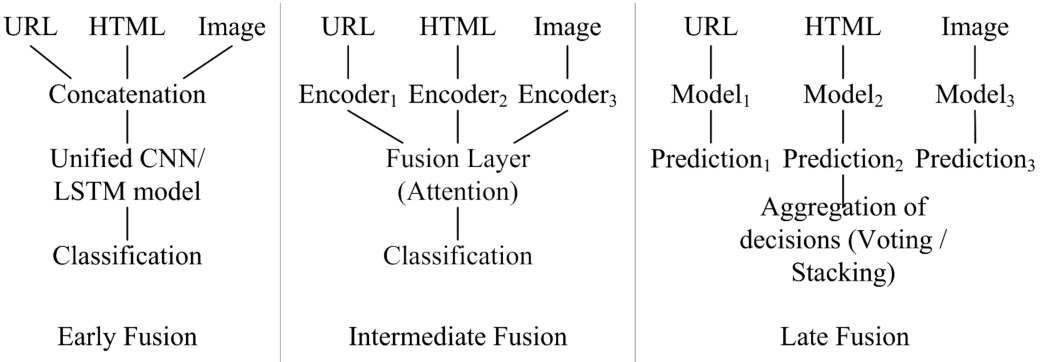


Figure 2. Modality fusion strategies.

2.4. Explainable Artificial Intelligence Methods for Detecting Phishing Attacks

Regarding the need for explainability (transparency, interpretation) of the decisions taken by phishing detection systems (i.e., explaining why a particular website is considered legitimate or phishing), a large number of studies have been published in this field of explainable AI research (XAI).

Explainable AI methods are classified according to the range of explanations (global and local) and the degree of focus on the model used (model-agnostic and model-specific [39]) (Figure 3). Global XAI methods [39] provide a general understanding of the model's behavior across the entire dataset, including feature importance and decision rules. Local methods [40] explain a model's predictions by showing how features affect a specific sample.

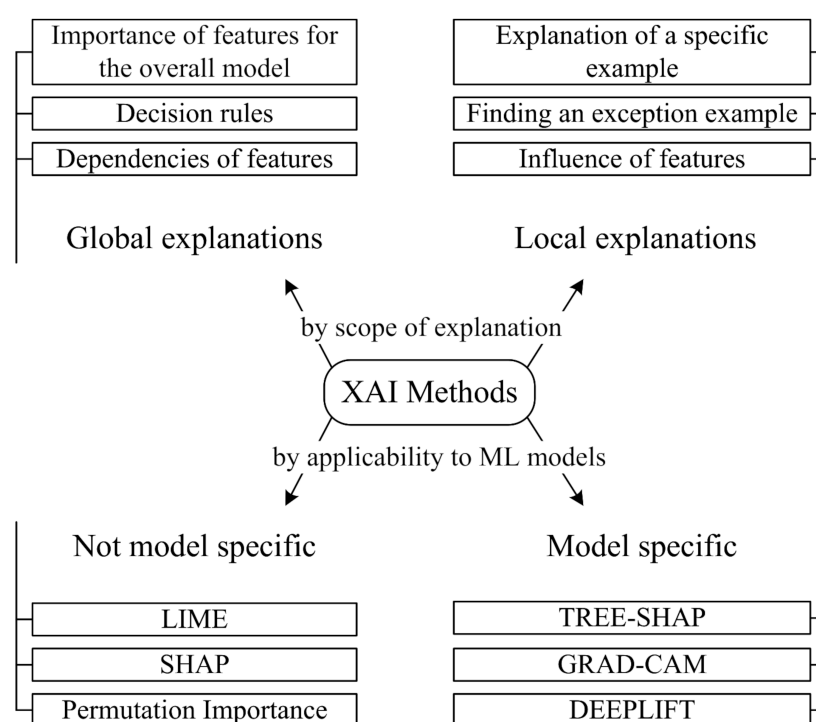


Figure 3. Classification of XAI methods to explain decision-making process in phishing attack analysis.

Most of the research on the use of XAI for phishing detection tasks is focused on combining ML and specialized XAI methods to explain the decisions that it helps to make. These XAI methods are [41] LIME, SHAP, decision trees, logistic regression, partial dependency plots, integrated gradients, and others. LIME and SHAP are the most popular.

The Local Interpretable Model-Agnostic Explanations (LIME) method is a model-agnostic interpretation method applicable to different types of ML models. The method explains individual predictions of the ML model (i.e., binary classification results) by generating biased input values and building a simple, interpretable “input–output” model based on the range of determined URL features. This allows one to identify the most important (informative) URL features that have the greatest impact on the classification outcome. LIME [40] constructs an interpretable neighborhood model in the vicinity of a prediction by generating biased samples and training a linear model. For example, the hybrid approach based on SHAP and LIME proposed in [33] increased the accuracy in the classification of phishing links for a random forest model to 97.06 percent (+0.65) and for an XGBoost model to 97.21 percent (+0.41).

Shapley Additive Explanations (SHAP) is an independent model-based method, as is LIME. It is based on game theory concepts and calculates Shapley values, which reflect the

contributions of each player (in this case, URL features) to the outcome of the cooperative game (in this case, an ML prediction of the nature of the website). SHAP [39] ensures local precision (the sum of the Shapley values is equal to the difference between the prediction and the average prediction), consistency (if a model change increases the contribution of a feature, its Shapley value does not decrease), and the absence of bias (features with no influence are assigned a zero value).

The following are examples of applications of this approach (ML + XAI methods).

- Phishing website detection system [16]: Base classifier model (ML)—random forest (RF); training datasets (datasets)—PhishTank and Tranco; number of used URL features—26; Lorenz Zonoids (multivariate extension of the Gini coefficient) was used as a feature selection procedure (XAI method).
- Phishing website detection system [42]: Base classifier models—SVM and RF; dataset—Ebbu 2017; number of URL features—40; XAI methods—LIME and Explainable Boosting Machine (EBM); prediction accuracy—94.7% (for SVM) and 97.3% (for RF).
- A system for detecting phishing websites [43]: six ML models were studied (XGBoost, LightGBM, RF, kNN, Twin SVM, CNN) using four datasets (ISCX-URLs, P.L-URLs, Phish Guard URLs, Suspicious-URLs) and the XAI-LIME method; the XGBoost model showed the highest forecast accuracy of 96.8%.

One approach to building a phishing website detection system based on XAI, proposed in [44], involves using a large language model (LLM) combined with processing input text data (URL features) using NLP, the Chain-of-Thought query decomposition algorithm, and a one-shot learning procedure (with a single example—a prompt). Here, the LLM simultaneously functions as a classifier and an explanatory module that generates a textual explanation of the classification result. Comparative experiments conducted by the authors [44] to evaluate the effectiveness of various LLM usage options (five LLM architectures and three datasets) showed the advantage of the GPT-4 Turbo Transformer—the F1-score was 0.92.

In [40], the authors present the integration of XAI with large language models to generate natural language explanations for the analysis of phishing emails. The system includes a detection module using classical ML models, which allows accuracy of 98.4%; a feature importance module (computing LIME and SHAP); an explanation module (DeepSeek v3, 671B parameters); and prompt templates with four explanation modes (detailed, educational, technical, and simplified). The explanation quality metrics on the reference set prepared by the authors showed accuracy of 94.2%, completeness of 87.6%, consistency of 96.8%, and actionability of 82.1%.

In [35], the authors introduced the PhishTransformer model, which showcases the efficacy of self-attention mechanisms in URL analysis. The model accepts a URL as input; following tokenization and the implementation of convolutional layers at the character level, positional encoding and self-attention layers are used. In an experimental dataset comprising a balanced collection of 100,000 URLs, the authors achieved an F1-score of 0.99. The attention mechanism allows one to identify key tokens in a URL, based on which the decision is most often made about whether a link belongs to a legitimate (official TLD (.com, .org), known brands) or phishing (“verify”, “secure”, “account”, “update”, “confirm”) website.

Across the scenarios previously discussed, the application of XAI approaches provided logical explanations supporting the selection of a particular solution, thereby identifying the URL features that most strongly affected the prediction result (i.e., the reasons for classifying a specific website as either legitimate or phishing), which is of significant importance to the user, who is often an information security specialist.

It should also be noted that, besides the mentioned approach to detecting phishing sites based on URL feature analysis, there are known examples of solving this problem using other features of website maliciousness and other methods of interpreting the results obtained with their help. For example, to analyze the source code of a web page, the bimodal Transformer model CodeBERT [45] can be used, pretrained on a large “code documentation” dataset, which allows the generation of text comments on the program code based on a “zero-shot” scenario, i.e., without fine-tuning the model parameters. Such information is of interest to information security specialists, since the elements of the web page source code provide insight into the structure, design, and functionality of the website.

It is advisable to use XAI methods such as Gradient-Weighted Class Activation Mapping (Grad-CAM) [46] and Deep Learning Important Features (DeepLIFT) [47] when analyzing visual features (website homepage images, screenshots, logs, brand indicators) using convolutional neural networks (CNNs).

The Grad-CAM method [48] creates heatmaps showing image regions that are important for classification using the gradients of the final convolutional layer.

The Phishpedia system proposed in [36] uses Grad-CAM to visualize critical regions on phishing web pages, detecting 1704 phishing sites in 30 days (1133 zero-days—66.5%) with a false positive rate of 1.4%. Grad-CAM detected fake bank logos, suspiciously placed login forms, and non-standard brand color schemes.

A second widely used method of interpreting CNN outputs (DeepLIFT) evaluates the influence of each input (image pixel) on the neural network output using backpropagation, considering the contributions of each feature to neural network activation.

The authors of [49] conducted a systematic comparison of ten CNN architectures on the Multitype Logo Phishing (MTLP) dataset for the visual analysis of screenshots of web resource home pages. To build neural network analysis models, the authors used transfer learning for subsequent fine-tuning on a balanced set of 15,000 examples.

The authors found that the deep NN DenseNetBlur121D provided the best balance between classification accuracy and model size (F1-score 94.39%, classification time 45 ms, size 28.1 MB); the NN MobileNetV2 was optimal (F1-score 87.6%, classification time 21 ms, size 9.2 MB) for offline use on mobile devices (edge deployment); and the NN ResNet50 demonstrated stability (F1-score 91.04%, classification time 38 ms, size 97.5 MB) on heterogeneous data.

In [37], the authors address the problem of detecting previously unknown (zero-day) phishing websites through visual similarity by training a deep CNN with a triplet-based loss function. The model achieved 81% accuracy on zero-day attacks, outperforming VGG16 (51.32%) and ResNet50 (32.21%). Key advantages of the proposed solution include the ability to train on a few examples (few-shot learning) and robustness to design variations (color palettes, element placement).

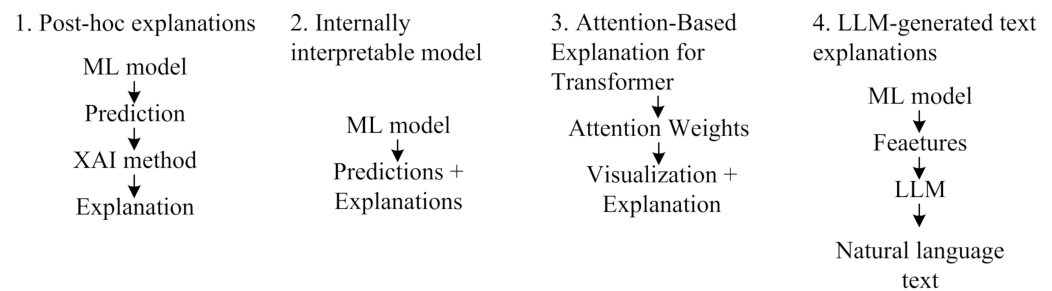
Table 5 and Figure 4 provide the characteristics of the explainable AI methods applied to the analysis of phishing attacks.

Regarding Table 5, local visual explanation allows for the visualization of image regions that are important for decision-making in terms of models. “Intrinsic” (internally interpretable) means that the model is inherently interpretable and does not require additional explanation methods.

Another important principle for constructing explainable AI systems is the level of integration between XAI and the ML model. A possible classification system for such integration is shown in Table 6.

Table 5. Summary of explainable AI methods.

Method	Type of Explanation	Applicability	Clarity	Accuracy	Application in Phishing
SHAP	Global + Local	Model-agnostic	Average	High	XGBoost, RF—URL features
LIME	Local	Model-agnostic	Average	Average	Any models—URL, HTML
Grad-CAM	Local visual	CNN-specific	High	Average	ResNet, VGG—screenshots
DeepLIFT	Local	DNN-specific	Low	High	CNN, LSTM—all modalities
Explainable Boosting Machine (EBM)	Intrinsic	Special model	High	High	Email phishing
Attention Mechanisms	Intrinsic	Transformers	Average	Average	BERT, Transformers—text
LLM text	Post-processing	Any	Very high	High	All modalities with context

**Figure 4.** Options for integrating XAI with ML models for analysis of phishing attacks.**Table 6.** Comparison of XAI integration levels.

Level	Methods	Difficulty of Implementation	Cost of Inference	User-Friendliness	Application in Industrial Systems
Post hoc	LIME, SHAP, Grad-CAM, DeepLIFT	Average	Low	Technical	Wide
Intrinsic	Decision trees, EBM, logistic regression	Low	Very low	High	Moderate
Attention	Transformers, BERT family	High	Average	Average	Growing
LLM text	On-premises and cloud LLM	Very high	High	Very high	Experimental

2.5. Main Datasets Used to Train Multimodal ML Models for Phishing Attack Detection

Some key datasets for training multimodal ML models are shown in Table 7.

Table 7. Key datasets for training phishing detection systems.

Name	Year	Data Type	Volume	Balance of Classes (Phishing/Legitimate)	Sources	Peculiarities
PhiUSIIL	2024	URL + features	235,795 URLs	57%/43%	UCI ML Repository [31]	48 features, focus on the latest threats
TR-OP	2024	Web pages	10,000	50%/50%	USENIX Security	Manual validation, 440 brands [27]
Combined BERT	2023	Multimodal	800 K+ URLs, 18 K emails, 6 K SMS	52%/47%	HuggingFace	Cross-channel data [50]
PhishTank	Updates daily	URL	1.5 M+	Only phishing	UCI ML Repository	Community-driven, API access [31]
APWG	Quarterly	URL + reports	Varies	Only phishing	apwg trends report q4 2024	Industrial standard [2]
MTLP	2024	Screenshots	15,000	50%/50%	IEEE	Multitype logo phishing [49]
Alexa Top 1M	Archival	URL	1,000,000	100% legitimate	Kaggle	Baseline of legitimate sites [51]

An analysis of the datasets and reviewed studies provides insights into certain aspects:

- Phishing sites are accessible for an average of 4–8 h before being blocked, and datasets become quickly outdated;
- The ratio of legitimate URLs to phishing URLs ranges from 100 to 1 to 1000 to 1 in real traffic among corporate information systems;
- The percentage of annotation errors in community-driven datasets is quite high (up to 15%);

- Feature distributions are dynamic and change over time (concept drift).

2.6. Promising Multimodal Systems for Analysis of Phishing Attacks

A promising direction in developing systems for the detection of phishing websites involves integrating GO, LLM, and XAI methods. This approach utilizes a multimodal system architecture that combines (concatenates) modules designed to process and analyze diverse features of phishing websites, including URLs, HTML content, screenshots, visual images, and source code. This comprehensive architecture ensures high accuracy, completeness, and reliability in detection while simultaneously achieving a high degree of interpretability (explainability) for the forecast results, attributable to the multifaceted nature of the analysis and the consideration of various perspectives on the detection task (see, e.g., [52]).

The research outlined in [27] introduces the PhishAgent system, an agent-based framework for phishing detection that leverages multimodal large language models. The perception module is designed to analyze various data modalities: it processes URLs using tokenization and embeddings, extracts features from HTML via Document Object Model (DOM) parsing, and performs object detection and brand database matching for logos. The reasoning module is predicated on employing a cloud-based large language model (LLM) utilizing a Chain-of-Thought (CoT) prompting strategy. The system implements the construction of a brand knowledge graph and a database of known phishing patterns. The action module allows the generation of explanations and execution of queries to external APIs (VirusTotal, WHOIS). The system yields an F1-score of 93.7–96.2% on three datasets. Another distinctive feature is its resistance to adversarial attacks, with a decrease in accuracy of only 2.3% (the best result among the systems considered by the authors). Disadvantages include the high cost of inference (via the API of a cloud commercial LLM) and a processing delay of 5–15 s per query.

The paper [28] presents the KnowPhish system, which addresses the problem of reference-based phishing detection without a predefined list of brands through the automatic construction of a multimodal knowledge graph. A multimodal approach is used, including a combination of visual (CLIP embeddings), text (BERT), and structural (GraphSAGE) analysis. The system achieves an F1-score of 89.6% for zero-day attacks and 94.3% for known threats.

The paper [34] proposes a system that has the highest accuracy in classifying phishing resources among non-LLM approaches through the late fusion of specialized models. The system architecture includes a URL branch (character-level, CNN), another URL branch (vocabulary-level, Transformer), and an HTML branch (graph convolutional network for DOM tree analysis). This approach yielded an F1-score of 99.27% on a specialized dataset.

General-purpose LLM models can be nearly as successful as specialized phishing analysis models in terms of classification accuracy if the necessary prompt data are prepared, enabling web search capabilities, and they can explain the decision in natural language. In [44], the best LLM results for GPT-4 Turbo and Claude 3 included an F1-score of 0.92 in zero-shot prompt mode.

Domain-specific models for phishing analysis typically achieve over 95% F1-scores on data with a known distribution but have low generalization capacity. It was shown in [44] that URLNet and URLTran models, trained on a single dataset, experienced 10–30% decreases in their F1-scores on URLs from other sources because of data drift. Therefore, a combination of transfer learning approaches (retraining specialized high-speed models) on new data is necessary, along with models capable of explaining the classification results, providing meaningful support to the expert making the final decision.

2.7. Findings from the Systematic Review

The challenges encountered in the advancement of phishing attack detection systems involve the following.

1. Contemporary machine learning models exhibit susceptibility to adversarial attacks [53], while existing certified defences impose prohibitive computational overheads. This highlights critical research needs: (1) the development of scalable certified defences for deployment in production systems, (2) the implementation of adaptive adversarial training robust to unknown attack types, and (3) the creation of robustness benchmarks tailored to the domain of phishing attacks.
2. Systems based on large language models (LLMs) [26,27] demonstrate high classification accuracy; however, the latency inherent in their operation—specifically the 5–15 s delay in decision-making and explanations—presents a critical barrier to their utilization in real-time defence systems.
3. The most advanced detection systems for zero-day phishing threats attain recall metrics in the range of 87–94%. Existing systems consistently fail to detect 6–13% of unknown threats. To mitigate this vulnerability, actual research [26,28] efforts are focused on advancing few-shot learning, meta-learning, and anomaly detection to address these limitations.

Thus, multimodal phishing detection systems presently demonstrate the following properties.

1. Current hybrid architectures demonstrate good detection accuracy, achieving performance rates of 98–99.68%. The integration of large language models (LLMs) has yielded a significant advancement in zero-day detection, with recall metrics ranging from 87 to 94%, while also enabling the generation of natural language explanations. Additionally, explainable artificial intelligence (XAI) methodologies are utilized in a substantial portion (80%) of recent research.
2. The adoption of multimodal systems has resulted in an improvement in accuracy metrics ranging from 5% to 12%. It has been demonstrated that late fusion techniques produce better outcomes when dealing with heterogeneous modalities. Furthermore, agent-based LLM architectures constitute a new paradigm characterized by their integrated reasoning functionalities.
3. A critical challenge is to ensure model adversarial robustness. This problem is characterized by a 3–9% decrease in accuracy, a temporal decay effect (an 8–15% degradation in performance without overfitting), and high LLM latency, which imposes significant real-time operational constraints.

Summarizing the review results, the following research gaps can be identified. Existing multimodal solutions for detecting phishing web pages either focus solely on one or two modalities (e.g., visual similarity and logos in VisualPhishNet and subsequent works) or employ heavy, cloud-based multimodal LLM agents with access to external knowledge (PhishAgent, KnowPhish, etc.). This significantly limits their applicability in on-premise security operations center (SOC) environments with strict requirements concerning latency, inference costs, and data control. In conclusion, it should be noted that the existing literature offers practically no solutions that, at the same time,

- Utilize multiple heterogeneous modalities (URL and metadata, symbolic URL, HTML code, page screenshot) in a late-binding architecture featuring a trainable meta-classifier;
- Incorporate an end-to-end explainable AI (XAI) loop with modality-specific interpretation methods and the consolidation of natural language explanations via a local LLM;

- Are implemented as an industrial microservice prototype, integrated with SOC/TI/SIEM frameworks, and evaluated using both standard classification metrics and an assessment of explanation usefulness for real SOC analysts.

3. Design of a Multimodal Phishing Website Detection System Using Explainable Artificial Intelligence

The present section describes the design and implementation process of a phishing web resource detection system, as well as a series of experiments to evaluate the effectiveness of multimodal analysis.

3.1. Structural Diagram of the Web Resource Analysis System

The structural diagram of the proposed web resource analysis system includes subsystems that implement a comprehensive analysis of the target resource using a multimodal set of late-binding models, with the ability to generate detailed reports in natural language for the SOC expert (Figure 5).

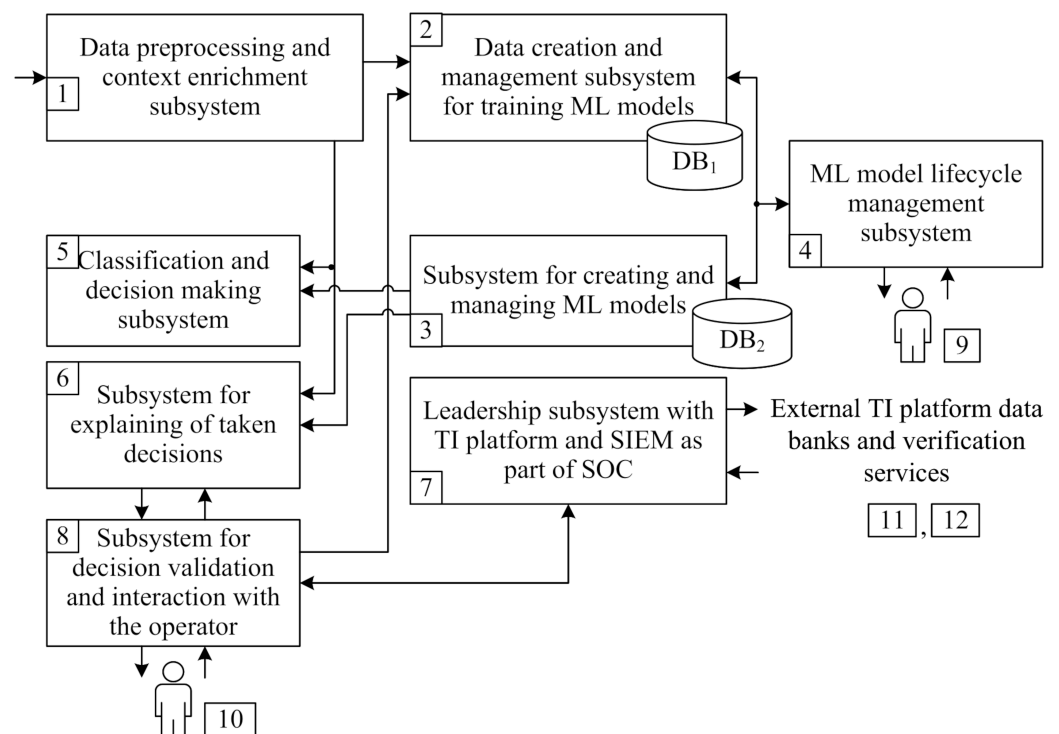


Figure 5. Architecture of the web resource analysis system.

The system receives the URL of the web resource being analyzed, downloads the HTML of the home page, and also downloads an image of the home page (screenshot).

Subsystem (1) for preprocessing initial data and context enrichment involves (Table 8)

- Feature Extraction;
- Context enrichment with data from external sources;
- Normalization, standardization, and the generation of a multimodal context.

URL analysis involves the extraction of lexical and statistical features and the collection of associated metadata. This metadata acquisition process includes verifying the URL domain's affiliation with known domain lists and acquiring domain registration age data (WHOIS), and other data are also collected. To enrich domain attributes, the system can utilize not only classic WHOIS queries but also the Registration Data Access Protocol (RDAP), which was standardized by the IETF in RFC 7480-7484 as a modern replacement

for WHOIS. Unlike the historically text-based WHOIS, RDAP provides structured responses in JSON format, a unified REST/HTTP query interface, and built-in security and access control mechanisms. This facilitates seamless integration with the system's microservice architecture and ensures compliance with contemporary registration data protection and minimization requirements (as stipulated by the GDPR and the current ICANN policy regarding the de facto replacement of WHOIS with RDAP).

Table 8. Modalities of analysis of the target web resource in the task of detecting phishing attacks.

No.	Multimodal Component	Data Type	Extracted Features (In Brief)
1	URL	URL components and domain features	– URL structure and statistical features (length, special characters, presence of IP instead of domain); – Lexical features; – Syntactic features; – Phonetic features; – Semantic features; – SSL/TLS certificates
2	HTML	Source code of the main page of the target resource	– HTML components; – DOM structure; – JavaScript code; – CSS
3	Image	Image of the main page of the analyzed resource	– RGB image 600 × 600 pixels
4	Metadata		– WHOIS information (domain registration) – Domain analysis (age, registrar); – Reputation databases (VirusTotal, Google Safe Browsing) – Threat intelligence feeds (PhishTank, URLhaus, OpenPhish)

Over 89 features are generated, providing comprehensive information about the URL.

The subsystem (2) used to create and manage data for the training of ML models enables one to supplement the database (DB1) with new labeled data from external sources (e.g., PhishTank subscription newsletters) and internal data validated by SOC specialists.

Subsystem (2) comprises the following components:

- A module dedicated to data labeling, relabeling, and label validation;
- A database (DB1) for storing enriched and prepared data;
- A version management module for dataset tracking and synchronization with corresponding machine learning (ML) models;
- A balancing and augmentation module configured to generate training, validation, and test datasets.

The regularly updated example database allows for the further training and/or retraining of classifier models as data drift occurs. A database of explanations for URL phishing indicators is also being developed (for example, by comparing them with MITRE tactics and techniques, known APT attacks, etc.).

The subsystem for machine learning (ML) model creation and management (3) facilitates the updating of operational models deployed within the data processing pipeline. This subsystem incorporates the following modules:

- Machine learning (ML) models specifically designed for the analysis of URLs/metadata, images, and the source code of target resource pages;
- A late fusion multimodal module that operates based on a weighted model voting mechanism;
- A product model database (DB2).

The structures of ML model interactions and the designations of specific models are shown in Figure 6 and Table 9.

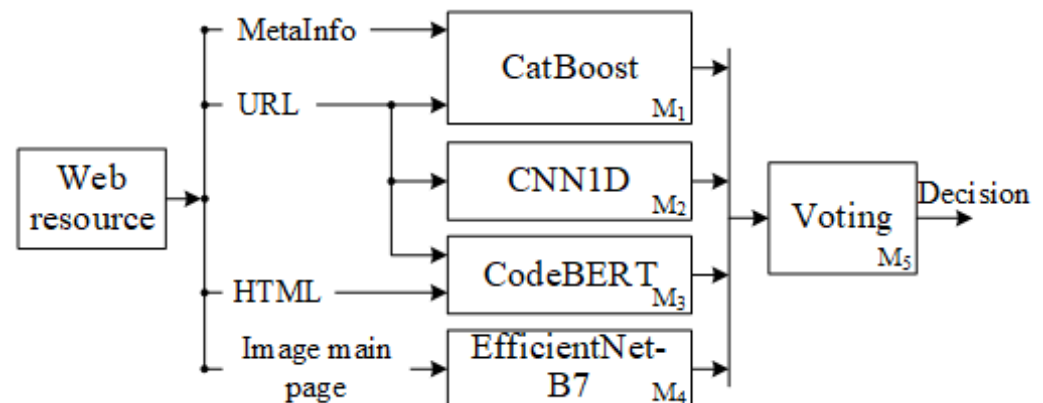


Figure 6. The structure of ML model interaction used for the analysis of phishing web resources.

Table 9. The main ML models used in the web resource analysis system.

No.	Multimodal Component	Model Designation	Model Type
1	URL + Metadata	M_1	Using the Optuna framework [54], hyperparameters of the CatBoost model [55] are selected in k-cross-validation mode with $k = 5$.
		M_2	CNN1D for character-level URL analysis.
2	HTML	M_3	Fine-tuning of the CodeBERT model [45].
3	Image	M_4	Fine-tuning of the EfficientNet-B7 model [56,57].
4	Late binding	M_5	Late binding of trained models using weighted voting.

Data analysts manage the model lifecycle using subsystem (4). Subsystem 4 includes the following modules:

- A performance monitoring module;
- A data drift detection module, which detects decreased model classification accuracy on freshly labeled data;
- The retraining and/or updating of model modules at specified intervals.

The subsystem for classification and decision-making (5), using trained models, classifies the web resource being analyzed. The subsystem for explaining the decision (6) being made implements the scheme presented in Figure 7 and Table 10.

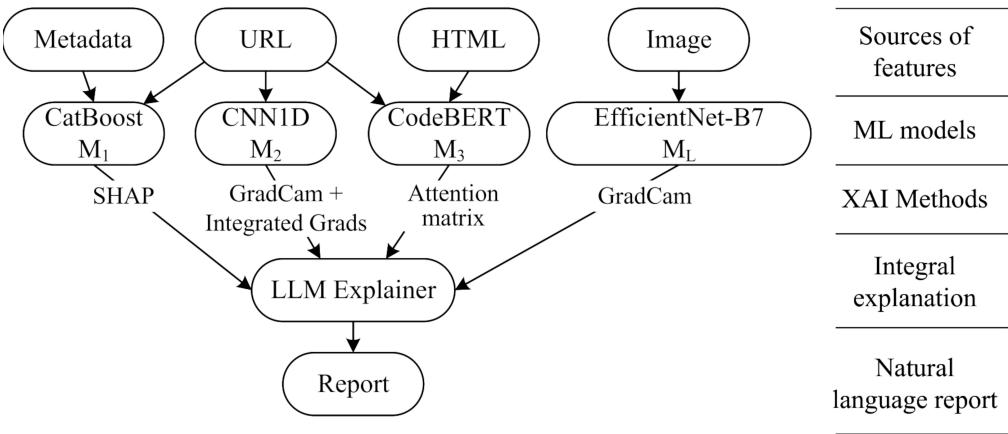


Figure 7. The process of analyzing a web resource and generating an explanation in natural language for an SOC specialist.

Table 10. XAI mechanisms used in combination with ML models as part of the web resource analysis system.

No.	Multimodal Component	Model Designation	Model	Formed Features of XAI
1	URL + Metadata	M_1	CatBoost	SHAP Values
		M_2	CNN1D	Grad-CAM, Integrated Gradients [58]
2	HTML	M_3	CodeBERT	Attention matrix [59]
3	Image	M_4	EfficientNet-B7	Grad-CAM

The subsystem implements API interaction with the locally installed LLM—the QwQ-32B reasoning model of the Qwen series [60]. Data with explanations for URLs’ classification as phishing links, prepared in subsystem (2), are used as a special prompt for zero-shot learning. The subsystem output (6) is a JSON file with a class name for the URL and a short text explanation of the result of the classification according to the proposed template. Only those websites that the model classifies as malicious are handled by the error correction mechanisms, which allows for acceptable overall performance of the system, given the significantly (by two orders of magnitude) lower LLM performance in terms of events per time.

The SOC’s thread intelligence and SIEM integration subsystem (7) collects metadata from web sources to improve the capabilities of modules for classifying and explaining information. This subsystem automates the response to phishing attacks by creating SOAR incidents and promptly reporting them to security monitoring specialists in the SOC.

The decision verification subsystem (8) provides an interface for first- and second-line SOC monitoring experts to adjust the marking. The subsystem includes the following modules:

- SOC analytics dashboard;
- Decision verification and data markup module.

Threat intelligence feeds (PhishTank, OpenPhish) and reputation services (VirusTotal, Google Safe Browsing, Kaspersky TI) serve as external data sources.

3.2. Key ML Models in a Multimodal Web Resource Analysis System

In the base version, ML models were trained independently of one another to implement a delayed binding strategy and to be able to select models for different scenarios (stream event processing or back-end analysis of individual websites).

3.2.1. Developing a URL Analysis Model

M_1 —CatBoost Model for Analysis of URL-Extracted Features and Metadata

This study tested several decision tree-based model organization options (random forest, XGBoost, CatBoost) for the analysis of tabular features extracted from URL and metadata in binary classification problems. Optuna has been used to select the optimal hyperparameters for each of the models. CatBoost scores 0.924 for the best ROC-AUC metric, indicating a strong ability to discriminate between classes; see Table 11.

Table 11. Comparative results of tree-based models on the prepared dataset.

Metric	CatBoost	Random Forest	XGBoost
ROC-AUC	0.924	0.915	0.918
Accuracy	0.910	0.890	0.900
Recall	0.880	0.850	0.860
F1-score	0.900	0.870	0.880

The CatBoost model provides the best representation of categorical and quantitative features with the ability to be efficiently multithreaded on the CPU. When translating the model into the C language and then compiling it using LLVM, the model's performance increased by 26% compared to the original version. To train the model, we used a composite sample of

- More than 150,000 domain names from real phishing campaigns (picked by PhishTank, OpenPhish, VirusTotal);
- 200,000 legitimate domains from Alexa Top and SSL certificates (Let's Encrypt, Mozilla).

M_2 —CNN1D for URL Analysis at the Character Level

CNN1D (Table 12) performs the character-level analysis of local URL patterns. The choice of a CNN was justified by the following:

- The ability to extract local patterns as substrings characteristic of phishing URLs (e.g., "login," "secure") through the use of convolutions with kernels of varying sizes;
- High analysis efficiency on the CPU because of the reduced number of model parameters, which allows for the simultaneous analysis of short ("@", " / /") and long ("login") URL anomalies, improving the classification accuracy.

A balanced (50% phishing and 50% legitimate web resources) MTLP dataset [61] was used to train the model. It included

- Phishing URLs (from open sources such as PhishTank and OpenPhish);
- Legitimate URLs (from popular services, including Google, Microsoft, and banking portals);
- Metadata—page source code, domain WHOIS data, screenshots, and binary labels ("phishing"/"not phishing").

The CNN1D model for web resource symbolic name analysis demonstrated high efficiency in detecting phishing URLs, achieving an F1-score of 91.41% on the test set.

Table 12. Parameters of CNN1D URL classification model.

Layers and Architecture Parameters	Description
Input layer—embedding	EMBED_DIM = 64 —converting characters to vector representation
CNN layers	Three parallel convolutional layers with kernels of sizes KERNEL_SIZES = {3, 4, 5}, number of filters NUM_KERNELS = 64
MaxPooling	Two layers
Output layer	Fully connected layer with sigmoid activation function
Loss function	Binary cross-entropy
Metrics	Accuracy, precision, recall, F1-score
Optimizer	Adam (LR = 0.001)
Data separation	The validation sample is 20% of the available data.
Regularization	Dropout (DROPOUT = 0.5) and early stopping (ReduceLROnPlateau).

3.2.2. Development of a Model for the Analysis of Visual Images of the Main Page of a Web Resource

*M*₄—CNN2D Model for Visual Image Analysis Based on Pretrained EfficientNet-B7

The analysis used a pretrained version of EfficientNet-B7. It has demonstrated high performance in the field of computer vision tasks because of its balanced depth and width and the resolution of the network. Pretrained in ImageNet, EfficientNet-B7 is able to effectively detect complex visual patterns such as logo distortions, non-standard layouts of UI elements, and other phishing indicators. To adjust the model for binary classification tasks, the last layer has been replaced by a linear classifier that estimates the probability that a web resource belongs to the class of “phishing”.

The model was trained on an MTLTP dataset containing phishing and legitimate website images (50/50 split). To increase the model’s robustness to data changes, an augmentation was used; the images were scaled to a fixed size (16:9 aspect ratio) and normalized using the ImageNet statistics.

For training, the BCWithLogitsLoss loss function was used. This function combines binary cross-entropy and logistic regression into a single layer, ensuring numerical robustness. The AdamW optimizer, with separate learning rates (1×10^{-5} for frozen layers and 1×10^{-4} for the classifier), minimizes the risk of weight corruption in the pretrained weights. OneCycleLR was used to dynamically adjust the learning rate, with parameters cycling through $[1 \times 10^{-5}, 1 \times 10^{-3}]$ depending on the number of epochs.

The developed visual image analysis model shows high performance in detecting phishing sites, with an F1-score of 93.22 percent.

3.2.3. Development of a Model for the Analysis of the HTML Code of the Main Page of a Web Resource

*M*₃—Transformer for Analysis of the HTML Code of the Main Page Based on the Pretrained CodeBERT

According to the source search, BERT variants are the predominant variants for the text analysis of HTML content. The CodeBERT neural network Transformer is pretrained in

code (six programming languages, including JavaScript) and is a powerful tool, including for JavaScript obfuscation detection.

The following were extracted from the source site's home page: JavaScript code blocks, inline handlers, and HTML structure (forms, iframes, meta-tags). The extracted blocks and URLs were combined into a single tokenization string with a maximum of 512 tokens.

The model was fine-tuned in batch mode on an MTLP dataset that contained phishing and legitimate web pages. The final classifier based on the CodeBERT retrained model achieved an F1-score of 0.950 using the test sample.

3.2.4. Development of a Meta-Classifer Model

Within the proposed architecture for a multimodal phishing web resource detection system, models M1–M4—specifically CatBoost for tabular URL features, CNN1D for symbolic URLs, CodeBERT for HTML code, and EfficientNet-B7 for visual page appearance—are trained independently. Each model generates probability estimates for an object belonging to the “phishing” class.

A subsequent level employs a trainable meta-classifier, M5, to execute a late fusion strategy. This architecture aligns with stacked generalization [62], where the second level (the meta-learner) takes the base model outputs as input and determines the optimal method for their combination.

The current implementation of the system incorporates a second-level tabular model (a linear classifier), functioning as a meta-classifier. This is realized as a generalized arbiter (stacking classifier) [62] that

- Utilizes estimations of class membership probabilities derived from the base models;
- Determines the weighting of modality contributions based on the validation sample to optimize the F1-score for the positive class (phishing);
- Facilitates operation with partially accessible modalities (for example, instances where screenshots are unavailable or HTML cannot be downloaded), which is essential for security operations center (SOC) environments.

Training for the M5 model is structured as a two-stage stacking process to prevent data leakage between layers.

- Description of the process of base model training. Models M_1 to M_4 are trained utilizing the original features of their respective modalities. For each object within the validation set, phishing probability predictions are generated by the four models.
- Creating the training set for model M5. For each URL, a meta-feature vector is developed comprising the M1–M4 predictions and consistency aggregates—specifically, the mean, variance, and the total number of models that identified the phishing class. The target labels are consistent with those specified in the original problem.
- Training a meta-classifier. A secondary tabular classifier is developed to optimize the F1-score for the positive class, utilizing the derived meta-features. Cross-validation is employed to select hyperparameters and the decision threshold; this facilitates a reduction in unidentified phishing attacks while simultaneously managing the proliferation of false positives.
- Inference mode on new data. During the operational phase, models M_1 to M_4 calculate probability estimates in parallel. These probabilities are subsequently provided to model M_5 , which generates the final estimate.

Within the framework of model ensemble theory [62], a trainable meta-classifier represents a more generalized case, where the weights and interactions between various modalities are determined according to the dataset. For the purpose of phishing detection, this approach is essential for several reasons.

1. Adaptive modality weighting. URL-based models occasionally generate false positives for popular legitimate domains (such as vk.com and rutube.ru), whereas a visual model correctly identifies these sites as non-phishing. A meta-classifier facilitates the downweighting of the URL branch in such instances—for example, when a high-confidence URL model prediction coincides with a low-confidence visual model output. Conversely, it enhances the contributions of those modalities that provide more reliable indicators for a specific class of examples.
2. Accounting for non-linear interactions between modalities. Stacking facilitates the consideration of complex scenarios—for instance, where a URL appears relatively innocuous yet the HTML code contains suspicious input forms and redirects, and the visual appearance mimics a brand logo. These combinations are characteristically found in phishing attacks. Simple voting mechanisms fail to distinguish between instances where “two moderately confident models agree” and those where “one model strongly disagrees while others remain uncertain”; conversely, a meta-classifier can be trained to assign distinct final decisions to these specific patterns.
3. Prioritizing the positive class (phishing) and managing the balance between false positives and false negatives. In the training of the M_5 classifier, the primary performance metric utilized is the F1-score for the “phishing” class. This approach facilitates a strategic shift in the balance towards minimizing false negatives (missed attacks) while maintaining an acceptable false positive rate, as previously established for the CatBoost model. Furthermore, the decision threshold for M_5 may be independently adjusted for positive and negative alerts, in accordance with specific SOC regulations.
4. Preserving functionality during modality unavailability. In a production environment, various data sources may periodically become unavailable; for instance, a screenshot might be absent, HTML code may fail to load, or integration with external threat intelligence (TI) services could be temporarily suspended owing to security policies. The meta-classifier can be trained on incomplete feature combinations—utilizing techniques such as modality masking or dropout—enabling it to transition seamlessly from full multimodal operation to a reduced configuration. Compared to a monolithic hybrid architecture, this strategy substantially increases the system’s fault tolerance.

3.3. Key XAI Technologies as Part of a Multimodal Web Resource Analysis System

In the design of the explanation subsystem, a deliberate decision was made to restrict its scope to post hoc and built-in XAI methods. These methods satisfy three essential requirements, formulated based upon the findings of the review (Section 2.4, Tables 5 and 6 and Section 2.7): (1) consistency with the specific model type employed, thereby ensuring the fidelity of the resulting explanations; (2) a predictable and acceptable computational overhead, requisite for stream processing within a security operations center (SOC) environment; and (3) demonstrable efficacy in the context of security and fraud prevention tasks. The chosen methodology is as follows: for the tree-based model M_1 (CatBoost applied to URLs and metadata), SHAP is utilized; for the convolutional models M_2 (symbolic URL analysis) and M_4 (screenshot analysis), the gradient-based methods Grad-CAM and integrated gradients are employed; and, for the Transformer model M_3 (CodeBERT applied to HTML), attention is harnessed as an intrinsic interpretation mechanism. The outputs generated by these disparate XAI components are subsequently converted into a unified intermediate representation and aggregated by a local large language model (LLM) into a concise text report, intended for the use of SOC analysts.

3.3.1. SHAP Coefficients for Interpreting the Performance of the M1 CatBoost Model

The CatBoost framework has an inbuilt implementation for extracting the values of SHAP coefficients for features. The results of the classification and decision-making test are shown in Table 13 and Figure 8 shows those for the synthetic test.

Table 13. Example of interpretation of the solution of model M_1 .

Synthetic Link	http://paypa1-security.tk/login		
features	Feature values	SHAP	Interpretation
domain_age_days	3	−0.450	A critical feature of a phishing resource
is_suspicious_tld	1	−0.380	An essential feature of a phishing resource
typosquatting_score	0.75	−0.315	A critical feature of a phishing resource

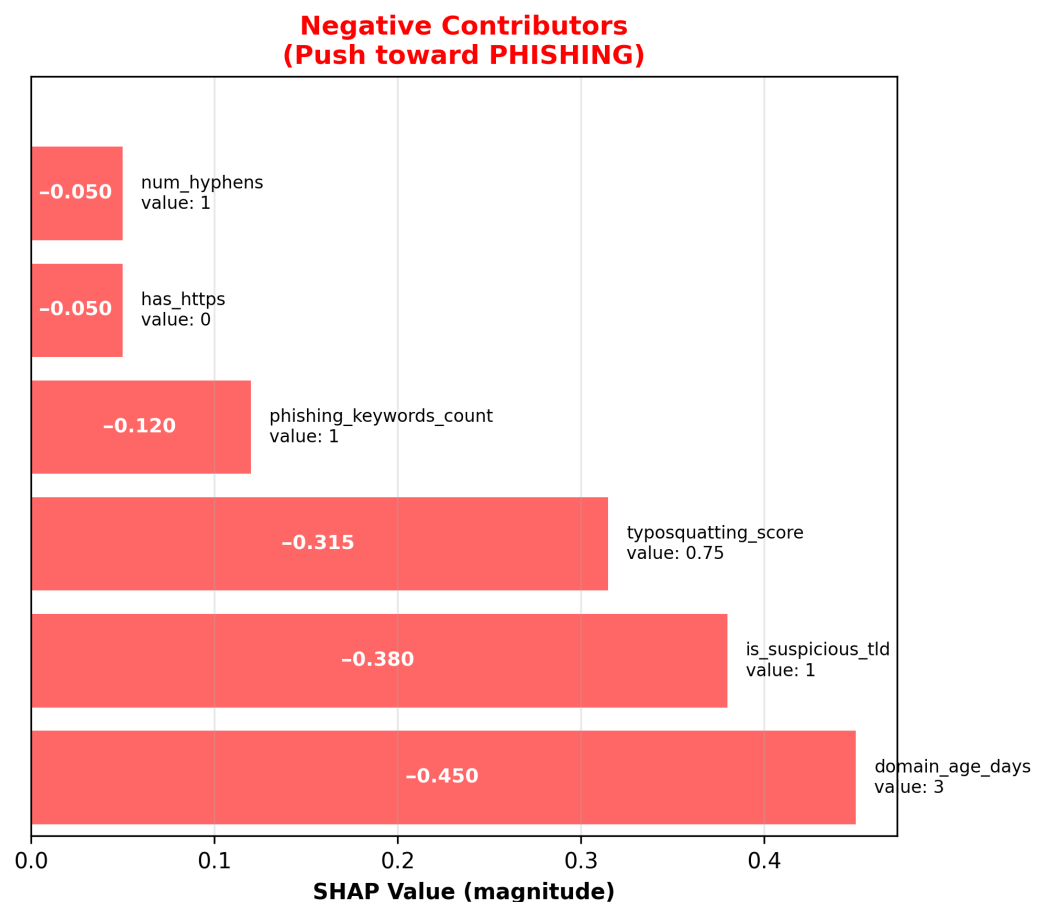


Figure 8. Histogram of top features determining the model's decision.

3.3.2. Grad-CAM and Integrated Gradients for Interpreting the Performance of the M2 CNN1D Model

An adapted version of the Grad-CAM method (for CNNs with one-dimensional input layers) weights the feature maps by importance using gradients and displays the focus area of the model in the intermediate representation (Table 14).

The integrated gradients method works at the input level (URL symbols) and indicates the contribution of each symbol to the model's final forecast based on an integrated gradient score (but it requires a defined baseline).

Table 14. Example of interpretation of the solution of model M_2 .

Model	Synthetic Example of a Phishing URL	Description
Grad-CAM	http://paypal-security.tk/login	The model focuses on the domain when deciding whether a resource belongs to a phishing resource.
Integrated Gradients	+0.9 +0.7 +0.95 +0.6 http://paypal-security.tk/login	The model refines URL symbols and their significance coefficients based on their contributions to the resulting solution.

3.3.3. Grad-CAM for Interpreting the Performance of the M_4 CNN2D Model

The EfficientNet-B7 neural network consists of mobile inverted bottleneck convolution blocks. The final layer prior to global average pooling was chosen for Grad-CAM operation. The MTLF dataset was previously used for the fine-tuning of the model, so a screenshot of the main page of the synthetic website, PayPA1-Security.tk, was used to illustrate the operation of XAI. The visualization results of the model’s operation are shown in Figure 9.

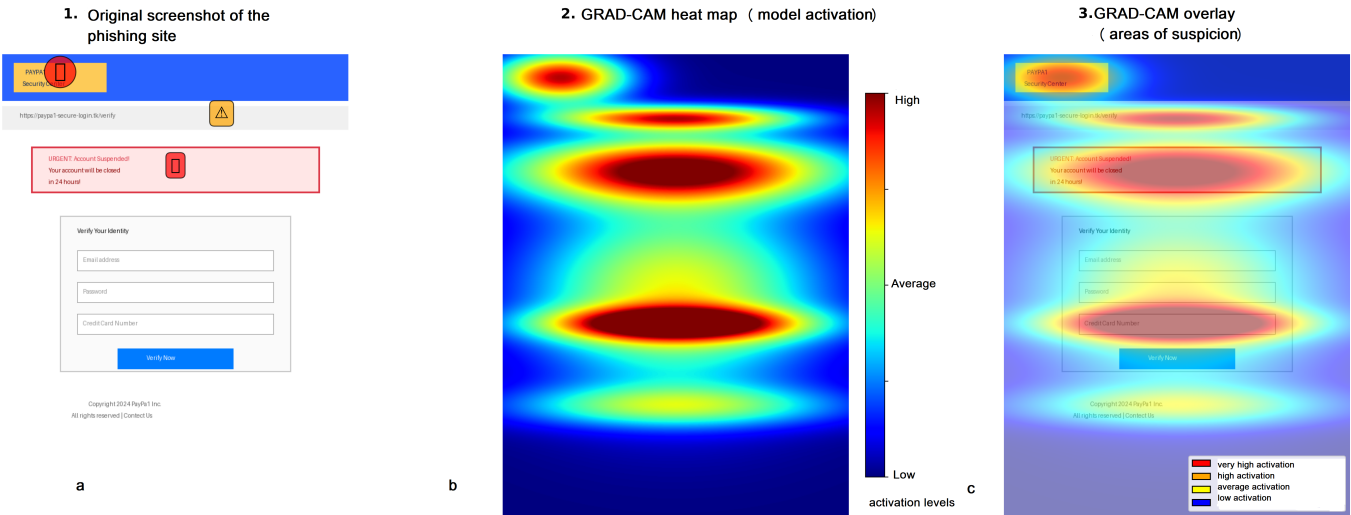


Figure 9. Illustration of Grad-CAM’s operation for the analysis of an image of the main page of a web resource: (a) screenshot with markers of areas indicating phishing; (b) heatmap of model activation; (c) overlay of the Grad-CAM heatmap on the original image.

3.3.4. Attention Matrix for Interpreting the Work of the M_3 Model, CodeBERT

The attention matrix is a table that describes which HTML page elements the model considers to be related. When interpreted, this approach allows the relationship between extracted page elements and the URL to be evaluated and the final decision to be made.

For the synthetic web resource <http://paypal-security.tk/login>, the HTML page size was 2985 characters, 232 tokens were extracted, and the attention matrix included 53,824 elements. Examples of extracted tokens are `<!DOCTYPEhtml >`, `<html lang = “en” >`, `<head >`, `<meta charset = “UTF – 8” >`, `<title >`, `PayPal, Security, Center`.

Figure 10 shows a histogram illustrating the distribution of attention values among the extracted tokens in the binary web resource classification task.

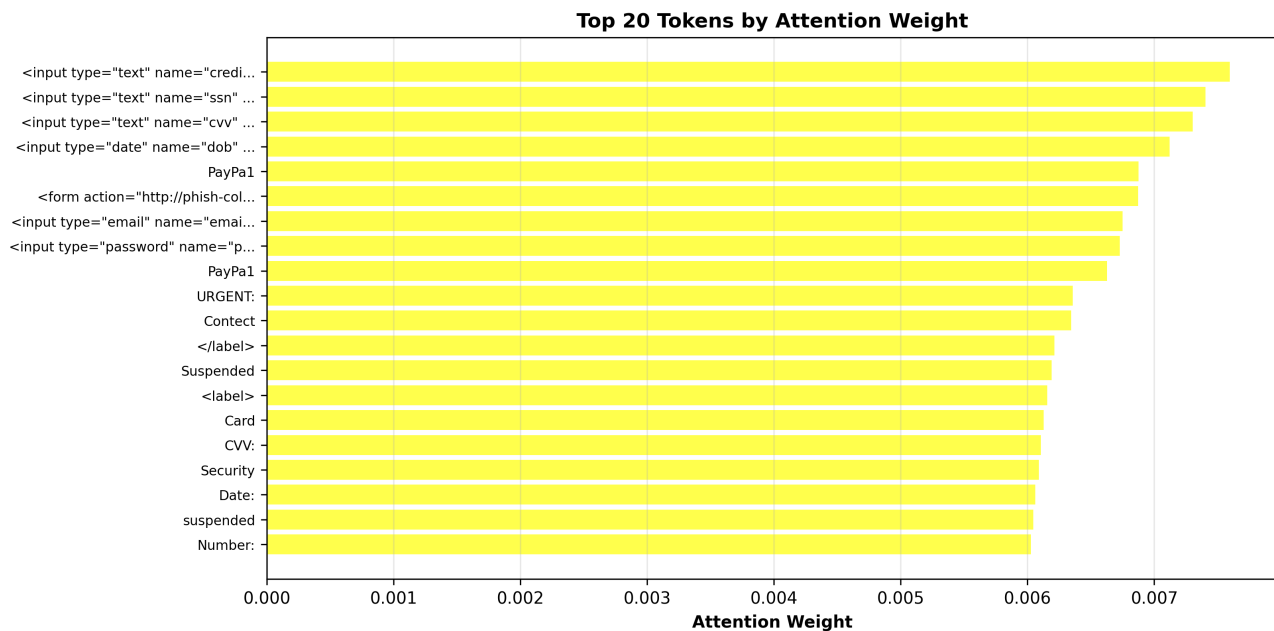


Figure 10. Most significant tokens based on the attention matrix analysis.

3.3.5. LLM Explainer for Preparing Final Reports in Natural Language

Local LLM Qwen2.5-32B is used to generate the final natural language report summarizing the explanations obtained by the different approaches.

The Grad-CAM results for CNN2D are prematched with elements of the HTML page and fed into the explanatory model as an “element”-“importance” dictionary. This is achieved by semantically dividing the page and mapping the visual areas to the DOM.

Other structured explanations obtained from XAI models are also fed into the template as data to populate the prompt template. The explanation module outputs a JSON file with a class label and a markdown document with an extended textual explanation of the classification result using a static template.

The LLM will generate a report (Appendix A) either for the websites of interest to the system user or only for sites that the model classifies as harmful, allowing for acceptable overall system throughput.

3.4. Software Architecture of a Web Resource Analysis System

The proposed algorithms and model for the analysis of web resources are implemented as software in Python 3.10 with an assessment of their effectiveness on real data (Figure 11).

The project consists of a group of containers, each of which performs a separate task, and their interaction provides complete coverage of the process of the analysis, verification, and notification of potential threats.

1. “Telegram bot” (App-tg_bot)—Implements the functionality of receiving incoming messages from users and serves as the first link in the value verification chain.
2. “Machine learning module”—A container with a machine learning model is deployed on the basis of the “m_service” image, where the “/check_domain” endpoint is implemented via the Flask API. When a POST request with a domain is received, multistage processing occurs.
3. “Threat intelligence module” (App-ti_service)—A container for integration with external reputation services such as Kaspersky TI, VirusTotal TI, and others via an API implemented on FastAPI.
4. “Data storage service”—A database for the long-term storage of all system operation results; a container with a MongoDB database is used.

5. Container for updating Elasticsearch (“App-update_es”)—In the test environment, the project uses Elasticsearch to analyze domains, where the “update_es” container is responsible for regularly updating the “idecoutm” index. Each time the script is run, a complete check is performed to see if the current domain has been processed before: if a match is found, the document is updated to reflect the new results, and, if not, a full check cycle is started. The update occurs every 3 s, which ensures that the data in Elasticsearch are up-to-date and enables the quick tracking of changes in the analyzed domains. With the help of Praeco (Elasticsearch alerting made simple; URL: <https://github.com/johnsusek/praeco> (accessed on 15 February 2025)), a correlation rule was created to notify analysts at the SOC, which allows for a prompt response to information security incidents.
6. Container management component (“Portainer”)—Uses the “portainer/portainer-ce:latest” image and provides a web interface for managing all of the project’s Docker containers. Special attention was paid during the design to safety and data isolation. All containers run in a single virtual segment, with no external ports exposed except for the Telegram API and the web administration interfaces. All requests are handled over HTTPS, logged, and may be signed for investigation.

Particular attention was also given to integration with existing monitoring systems during the design process. The used log structure allows for the transmission of events to SIEM systems, such as Wazuh, Splunk, and ELK-based solutions. The records are stored in MongoDB and replicated to Elasticsearch, which allows for the quick creation of analysis views. The index is updated automatically, and visualization is possible via Kibana or a customized dashboard in Praeco. This enables SOC analysts to monitor and make informed decisions about suspicious activity in real time.

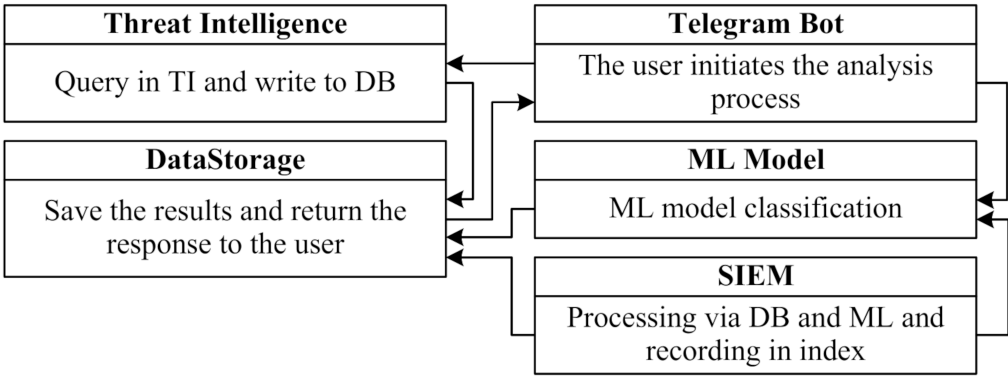


Figure 11. Architecture of interaction of software modules implementing key components of the system.

The system was adapted to operate in a real-world information security center, including with regard to response procedures. The decision algorithm may be based on machine learning models or on data from external threat intelligence services. If necessary, the administrator may manually change the status of an object by adding it to the whitelist or blacklist. Scenarios for automatic action following a threat detection are described separately, e.g., sending the result to an email gateway, adding an IP address to a blocking system, or sending a notification through a corporate messaging application.

The prototype was deployed using a high-performance server, the parameters of which are shown in Table 15. In the isolated environment, each model used dedicated GPUs and data were exchanged between the models through RAM channels.

Table 15. Server parameters for running of models.

Parameter	Characteristic
GPU	4 GPU Tesla V100
GPU video memory capacity	128 GB
CPU	Intel Xeon E5-2698 v4 2.2 GHz (20-core)
RAM capacity	256GB RDIMM DDR4

Thus, the developed architecture allows the following:

- Processing queries in real time;
- Storing results and metadata for subsequent analysis;
- Scaling individual modules independently (e.g., multiplying ml_service);
- Integrating with external sources via APIs and with internal security systems via SIEM interfaces;
- Providing a visual interface for both users (via Telegram) and administrators (via Portainer and Mongo Express).

4. Computational Experiment to Evaluate the Performance of the Web-Based Resource Analysis System

Several computational experiments have been performed on our own dataset and on the available MTLTP dataset to provide a comparative assessment of the capabilities of the proposed solution.

The key stage in developing the automatic phishing web resource detection system was the formation of an up-to-date dataset. Datasets widely used in research (ISCX-2016 Dataset, EBBU-2017 Dataset, HISPAR-Phishstats Dataset) quickly become outdated. At the same time, the quality of the classification model, its ability to identify new attack patterns, and its sensitivity and specificity directly depend on the correctness and relevance of the data.

4.1. Computational Experiment I on the Prepared Dataset for Models M_1 and M_3

4.1.1. First Phase of Testing: Collecting the Dataset (D_1) and Preparing Models

Data were collected from several authoritative sources (Dataset D_1):

- Phishing URLs were obtained from open repositories Phishing Site URLs (Kaggle), OpenPhish, URLHaus, and PhishTank, where malicious link databases are updated daily;
- Legitimate URLs were selected from the Alexa and Majestic Million Top 1 Million Websites lists, as well as downloaded from secure corporate proxy logs.

The initial sample in Table 16 contained over 4 million rows, of which, after cleaning and filtering, approximately 3.2 million unique URLs remained, balanced across classes.

Data labeling was performed semi-automatically. Some URLs were labeled based on the source (for example, if a link was taken from OpenPhish, it was a priori phishing). In complex cases, the following were used:

- Cross-checks via the VirusTotal API;
- A built-in reputational score checking module;
- Manual checks by WHOIS data (registration date, TLD zone, domain activity);
- Validation using OpenAI and QwQ-32B models in batch mode.

As a result, a dataset was formed containing

- 1.6 million phishing web resources;

- 1.6 million legitimate web resources.

Table 16. Estimated time costs for image processing in batch mode.

Model	Operation	Batch Performance Evaluation	Learning Mode/ Inference During Integration
M_3	HTML tokenization	25–50 ms	GPU/GPU
	CodeBERT ONNX	200 ms	GPU
	Attention extraction	15–20 ms	GPU
M_1	Feature extraction	Less than 8 ms per URL	GPU/CPU
	SHAP computation	Less than 12 ms per URL	GPU/CPU
	CatBoost C-compiled	Less than 8 ms per URL	GPU/CPU

Similarly, CatBoost shows the best ROC-AUC metric of 0.924, indicating a strong ability to discriminate between classes. For the best CatBoost model, the false positive rate metric was 19.98%, and the false negative rate metric was 8.66%. Given the comparable performance of the models based on ensembles of decision trees, the CatBoost model was selected, which had the best F1-score indicators on the test sample, but the preparation of the data preprocessing pipeline and the speed of model training were significantly higher. The model also allows for the evaluation of the significance of each feature during classification. The CatBoost model provides the best representation of categorical and quantitative features with the ability to be efficiently multithreaded on a CPU. When translating the model into the C language and then compiling it using LLVM, the model's performance increased by 26% compared to the original version.

Additionally, a classifier was built based on the retrained CodeBERT model, deployed in the ONNX model format in the FastAPI container. The ONNX model format is a static computational graph in which the vertices are computational operators and the edges are responsible for the sequence of data transfer across the vertices, which allows the model to run 1.5–2 times faster in classification mode. However, using the model on a server with a CPU in multithreaded mode renders it inferior to models based on CatBoost.

The peak performance estimates for the number of requests processed in the security operation center (SOC) were as follows: 231 thousand per day, 11 thousand per hour, and 100 per second.

4.1.2. Second Stage of Testing—System Throughput Assessment

The second phase of the evaluation involved deploying the complete system on a test bed simulating the conditions of the SOC. The flow of incoming URLs was formed from several sources:

- Corporate mail gateway logs;
- Traffic through proxy servers and web filters;
- Specially generated requests through a Telegram bot.

The total volume of test samples was about 100 thousand URLs, arriving at an average speed of 500 requests per minute. Under this load, the system demonstrated stable operation, with an average processing time of 47 ms for one request, including preprocessing, calling the model, and writing the results to the database. The peak load was up to 1100 URLs/minute, while no timeout errors or microservice freezes were recorded.

4.1.3. Third Stage of Testing—Testing as Part of the SOC

During integration with Praeco, an attack was simulated: within 5 min, the system received 20 fake URL links using the secure-update-login[.]com subdomain and replacing Latin characters with Cyrillic. The system worked with high accuracy, generated a

trigger based on the correlation rule, and sent a notification to the SOC Telegram channel (Figure 12).

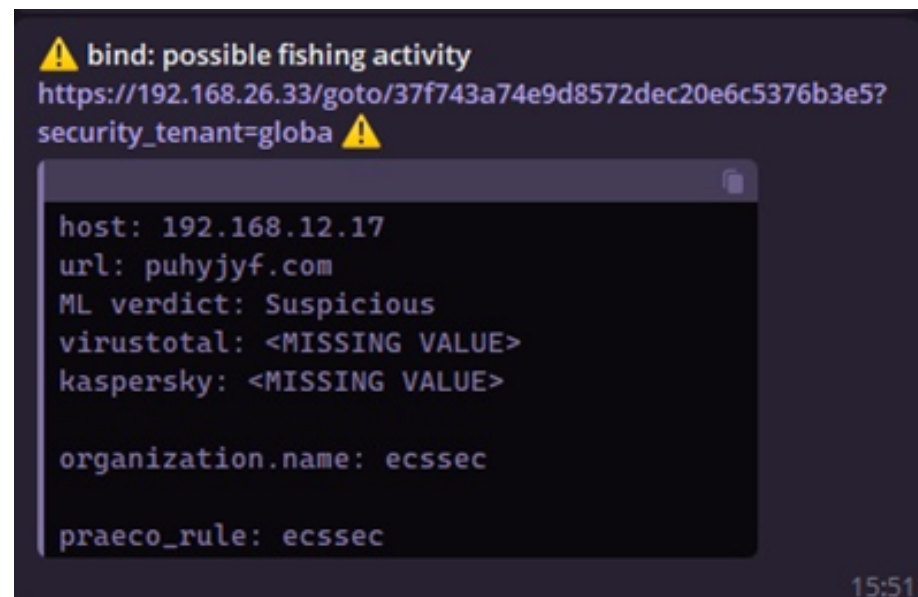


Figure 12. ML trigger notification.

4.1.4. Fourth Stage of Testing—Zero-Day Resource Testing

In addition, manual testing was conducted on “zero-day” links—URLs that were not previously included in the database and which imitated new phishing patterns. Of the 100 such links, the system identified 92 as suspicious, including six that had no signs in the VirusTotal databases but had an anomalous structure. For an objective assessment, the system was compared with open tools:

- Google Safe Browsing;
- The OpenPhish Detection API;
- The URLhaus API.

The proposed model detected 15–20% more “fresh” phishing links, especially in cases of obfuscated addresses, short redirects, and character substitution. A significant advantage was provided by the explainability of classifications—the SOC analyst could immediately see on the basis of which features the link was classified as phishing.

4.1.5. Testing the Subsystem for the Explanation of Positive Responses

The application of a subsystem to explain the reasons for classifying resources as phishing is shown for the expanded QwQ-32B model. A detailed explanation is used to justify a positive classification for SOC first-line monitors and allows them to agree or mark the item as a false positive for the further adjustment of the training data. The application of approaches to estimating the SHAP coefficients at this stage was carried out only at the stage of preparing data for model training, since the obtained results require additional interpretation for monitoring specialists.

Thus, the conducted testing confirmed not only the high accuracy of the model but also its suitability for deployment in real time. The system is capable of analyzing a large flow of incoming URLs, is scalable, and provides integration with response infrastructure, making it applicable to monitoring centers, highly secure organizations, and even national CERT structures.

4.1.6. Summary Results of Computational Experiment I

Table 17 presents the summary results of computational experiment I.

Table 17. The results of computational experiment I (M1, M2, M3).

Experiment	Model (Designation)	Model (Name)	Dataset	F1-Score
I	M1	CatBoost	D1 (own dataset)	0.900
	M3	CodeBERT		0.950
	M1 + M3	CatBoost + CodeBERT + Voting Classifier		0.972

4.2. Computational Experiment II on the Prepared Dataset for Models M_2 and M_4

A balanced (50% phishing and 50% legitimate web resources) MTLP dataset was used to train the models (Table 18).

Table 18. Estimated time costs for image processing in batch mode for computational experiment II.

Model	Operation	Batch Performance Evaluation	Learning Mode/Inference During Integration
M_2	Tokenization	3 ms	GPU/GPU
	CNN1D forward pass	7 ms	GPU
	Grad-CAM + integrated gradients	55 ms	GPU
M_4	Screenshot capture	250 ms	GPU/CPU
	Image preprocessing	15 ms	GPU/CPU
	EfficientNet-B7 inference	200 ms	GPU
	Grad-CAM computation	150 ms	GPU
	Region analysis	50 ms	GPU/CPU

The CNN1D model for web resource symbolic name analysis demonstrated high performance in detecting phishing URLs, achieving an F1-score of 91.14% on the test set (Table 19).

The fine-tuned EfficientNet-B7 model for visual image analysis demonstrated high performance in detecting phishing websites, achieving an F1-score of 93.22% (Table 19).

Table 19. Summary results of computational experiment II.

Experiment	Model (Designation)	Model (Name)	Dataset	F1-Score
II	M_2	CNN1D	D2 (MTLP)	0.914
	M_4	EfficientNet-B7		0.932
	$M_2 + M_4$	CNN1D + EfficientNet-B7 + Voting Classifier		0.944

4.3. Computational Experiment III on the Prepared Dataset for All Models

The final experiment assessed the multimodal performance of the system on datasets D_1 and D_2 . The results are shown in Table 20.

Table 20. Summary results of computational experiment III.

Experiment	Model (Designation)	Model (Name)	Dataset	F1-Score
III	$M_1 + M_2 + M_3 + M_4$	CatBoost + CodeBERT + CNN1D + EfficientNet-B7 + Voting Classifier	D_1 (own dataset)	0.989
	$M_1 + M_2 + M_3 + M_4$	CatBoost + CodeBERT + CNN1D + EfficientNet-B7 + Voting Classifier	D_2 (MTLP)	0.953

4.4. Exploratory Assessment of Robustness of a Multimodal Architecture Against Conventional Obfuscation Techniques

In addition to the primary computational experiments, an exploratory assessment was conducted to evaluate the robustness of the multimodal architecture against conventional obfuscation techniques frequently employed in phishing campaigns. Subsequently, several subclasses of “enhanced” attacks were synthetically generated, utilizing the initial corpus of URLs as a baseline (Table 21).

Table 21. Assessing the system’s resistance to obfuscation techniques.

Nº	Enhanced Attack Subclass	Decrease in Performance of Individual Models	Decrease in Performance Quality of Multimodal Model
1	URL obfuscation (the use of homoglyphs, the insertion of superfluous subdomains, the addition of suffixes or prefixes, and the substitution of domain names with IP addresses).	2	1.5
2	HTML obfuscation (the injection of “noise” into the DOM tree, the fragmentation of key strings, and the use of JavaScript-based encoding).	4	2
3	Visual camouflage (modifying the color scheme and non-essential template elements while preserving the fundamental structure of the login form).	5	3

For each subsample, a comparison was conducted between the performance degradation of individual branches—specifically URL, HTML, and visual—and that of the integrated multimodal framework employing a meta-classifier.

Preliminary results indicate that, for URL and HTML obfuscation, unimodal models experience a reduction of up to 4–7 percentage points in the F1-score; in contrast, the multimodal system demonstrates a more modest performance degradation, owing to the complementary nature of the integrated modalities. A comprehensive and systematic investigation into the robustness against targeted adversarial attacks remains the objective of future research.

4.5. Evaluating the Quality of Explanations Provided for Resulting Decisions

During the assessment of the proposed system’s performance, a deployment scenario was evaluated on a test stand designed to simulate the operational conditions of a security operations center (SOC). Within this phase, a sample of 100 URLs was generated, all of which were classified as “zero-day” threats for the period under analysis. These URLs lacked an established negative reputation in public sources but were deemed potentially fraudulent or malicious based on internal analysis and retrospective incident reviews. This sample was processed by the system as an event stream, with the test stand replicating the standard workflow of an SOC analyst: the reception of events, the analysis of correlation rules and detector triggers alongside their corresponding explanations, and the formal recording of decisions.

The second substage was dedicated to evaluating the efficacy of the decision explanation subsystem. To assess these explanations, 30 unique cases—generated by the proposed system for specific URLs—were selected at random. For each instance, the system produced a textual explanation delineating the primary features that influenced the classification. These 30 explanations were initially appraised in their basic modes (without the RAG subsystem) by three active first-line SOC analysts. The experts were asked to rate the utility of each explanation on a five-point scale: 1—“explanation useless”; 2—“minimal assistance”; 3—“somewhat useful but requires significant refinement”; 4—“useful for decision-making”;

and 5—“highly useful, satisfying the analyst’s requirements almost entirely”. This process generated a total of 90 ratings. The mean score for the URL dataset was 3.97, with the majority of ratings concentrated within the 4–5 range. On average, the experts perceived the explanations as useful, although they noted scope for further enhancement regarding completeness and structure.

In the subsequent stage, the RAG subsystem for the analysis of cyber-intelligence data [63]—previously developed by the authors—was applied to the same 30 events. This subsystem was integrated with a knowledge base encompassing CTI reports, descriptions of offensive techniques, and historical incident records. Explanations were subsequently re-generated to incorporate the identified context; consequently, the system began to reference established phishing methodologies, cite examples of analogous campaigns, and highlight correspondences with specific tactics and techniques (for instance, utilizing the MITRE ATT&CK framework). The three experts then re-evaluated the utility of these “augmented” explanations using the same five-point scale. This second round of assessment yielded a further 90 ratings (Table 22). The mean score for the utility of explanations within the RAG-enabled mode increased to 4.2. Simultaneously, the distribution of these ratings shifted markedly: the proportion of five-point scores rose, while one- and two-point ratings were entirely eliminated. This shift reflects a significant reduction in the number of explanations perceived as “insubstantial” or of limited practical value.

Table 22. The distribution of points across the two explanation evaluation scenarios.

Parameter	Explanation Subsystem without RAG	Explanation Subsystem with RAG
Distribution of points	2 points — 3 scores; 3 points — 9 scores; 4 points — 66 scores; 5 points — 12 scores	2 points — 0 scores; 3 points — 9 scores; 4 points — 54 scores; 5 points — 27 scores
Average points	3.97	4.20
Fleiss’s κ	0.82	0.84
Cronbach’s α	0.90	0.96

Particular attention was paid to the consistency of the expert assessments. Utilizing the data compiled for both operational modes, Fleiss’s κ was calculated as a measure of inter-rater agreement for an ordinal scale, alongside Cronbach’s α to determine the internal consistency of the rating scale. For the baseline mode (without RAG), involving 30 explanations and three experts, Fleiss’s κ was 0.82, and Cronbach’s α was 0.90. These results indicate a high degree of consistency: the experts generally concurred in categorizing the explanations as “rather useful” or “useful”, with only localized instances of disagreement. In the RAG-enabled mode, the consistency metrics showed further improvement, with Fleiss’s κ reaching 0.84 and Cronbach’s α rising to 0.96. Consequently, the integration of RAG not only enhances the mean utility score of the explanations but also ensures more homogeneous expert evaluations; analysts are more inclined to perceive the same structural merits within the explanations and are less likely to diverge in their assessments of their practical value.

In the context of the experiment, mitigating hallucinations demands a reduction in instances where the explanation subsystem

- Invents non-existent facts (non-existent organizations, IOCs, tactics, and techniques);
- Incorrectly associates URLs with techniques/tactics;

- Generates explanations that do not logically follow from internal features and the retrieved context.

In the initial version of this work, the explanation subsystem leveraged an extended context generated by the XAI algorithms. The expert evaluation results indicate that, for general-purpose large language models (LLMs), this information is typically sufficient to generate a structured explanation with an acceptable standard of quality.

When utilizing RAG, the quality of explanations is enhanced through the incorporation of a specialized cyber-intelligence knowledge base [63].

Collectively, these findings demonstrate that the system exhibits high sensitivity to zero-day attacks owing to the structural analysis of URLs. Furthermore, the explanation subsystem—particularly when integrated with the RAG component—is perceived by SOC analysts as a practically valuable decision support tool, characterized by a high degree of inter-expert agreement.

4.6. Discussion

The implemented prototype of the web resource analysis system validated the efficacy of a multimodal approach, integrating URL analysis, visual image, and HTML code analysis. However, empirical testing identified several critical limitations that adversely affect the user experience and compromise the perceived system reliability.

The URL analysis model occasionally misclassifies legitimate and highly popular web resources—for instance, vk.com and rutube.ru—as phishing sites. These domains lack obvious suspicious features, such as lengthy query parameters or hidden iframes. These misclassifications may be because of the coincidental structural matching of these URLs with patterns recorded in the training dataset. A similar challenge is observed with the PhishLang model; even high performance metrics (an F1-score of 0.94) do not entirely eliminate classification errors on legitimate sites, likely because of the limited diversity of the training data.

The generation of web page screenshots with arbitrary resolutions and aspect ratios that do not align with the model's training parameters results in degradation in visual analysis quality. This occurs because the EfficientNet-B7 model, which was trained on standard data, loses its capacity to recognize key visual features, such as distorted logos.

A significant advantage of the multimodal approach is its capacity to yield objective assessments, particularly when faced with false positives originating from a URL analysis model. For instance, in an evaluation of popular websites such as vk.com and google.com, the convolutional neural network (CNN) model erroneously classified them as phishing sites. However, the EfficientNet-B7 model, utilizing visual content analysis, accurately identified these sites as legitimate. This discrepancy underscores the robustness of the multimodal framework in mitigating the systematic errors of individual constituent models, thereby substantially reducing the overall risk of false blockings.

The implemented late binding strategy was necessitated by the pronounced heterogeneity of the modalities and models utilized. The analytical branches—incorporating CatBoost, CNN1D, CodeBERT, and EfficientNet-B7—operate within disparate feature spaces and are subject to varying constraints regarding input length, formatting, and resource requirements. An attempt at early or intermediate fusion would have required the construction of a unified common representation and a monolithic architecture featuring cross-modal attention; such an approach would have significantly complicated system training and maintenance without guaranteeing performance gains over independent, specialized models. This approach is consistent with research into multimodal training involving missing modalities, where late or hybrid binding confirms enhanced robustness to data omissions [64].

The late binding strategy ensures system resilience in the presence of partially observed data. The availability of modalities is inconsistent: HTML code and screenshots may be absent because of security policies or failures in web page parsing and rendering, while external threat intelligence (TI) services may be subject to temporary outages. Through late binding, a decision is formulated based on the subset of modalities for which features have been successfully extracted. Conversely, in early fusion architectures, the absence of a single modality necessitates either the complex simulation of missing input data or the total abandonment of event processing. This distinction is particularly critical for SOC, where the loss of events because of incomplete data is unacceptable.

In a real-world SOC, both performance metrics and event processing latency are critical considerations. Branches dedicated to URL and metadata analysis (CatBoost, CNN1D) ensure request processing within a few to tens of milliseconds; conversely, HTML and screenshot analysis based on CodeBERT and EfficientNet-B7 are considerably more resource-intensive. A late binding strategy permits the customization of the utilized modalities to suit specific operational scenarios, ranging from real-time detection to the in-depth investigation of incidents using historical data. This approach enables the generation of a baseline forecast via high-speed branches, while “heavyweight” models are reserved to further enrich only the most suspicious events—a level of flexibility that is unattainable with early binding architectures.

A late binding architecture, incorporating a trainable meta-classifier, facilitates the dynamic control of active analytical branches; specifically, certain models may be temporarily suspended during periods of increased workload or limited computational capacity, while the system’s fundamental functionality is maintained by the remaining modalities. This inherent modularity, alongside the capacity to disable resource-intensive models, is identified in [65] as a primary advantage of late binding for industrial systems operating with heterogeneous signal sources.

A further distinction between the proposed architecture and conventional late binding schemes lies in the employment of a trainable meta-classifier as an alternative to simplistic voting or probability averaging. This meta-classifier receives vectors of output estimates—such as probabilities or logits—from the base models and is trained as a discrete, lightweight model, thus implementing the instance-wise weighting of modality contributions. This strategy, analogous to ensemble stacking, facilitates the partial modeling of inter-modality dependencies at the decision level. Consequently, it enhances the system’s robustness against noise or systematic errors within individual branches and improves the overall performance metrics relative to both isolated modalities and their unweighted fusion. Comparable observations regarding the advantages of this specific late binding implementation are presented in [66] within the context of unified multimodal fusion schemes.

The model lifecycle and the management of data drift deserve special attention. As the architectural branches are decomposed by modality, each constituent model may be retrained on novel data or replaced with a more contemporary architecture without necessitating the concurrent retraining of the entire multimodal ensemble. This is especially pertinent to the detection of phishing and malicious activity, where training datasets depreciate rapidly and novel attack patterns emerge continuously. Experience in multimodal analysis within related domains [67] indicates that late or hybrid binding facilitates the controlled updating of individual modalities and the localization of drift without requiring a comprehensive system overhaul.

The chosen late binding strategy facilitates the implementation of explainability and interaction with the LLM-based explanation subsystem. For each branch, specialized interpretation methods that generate modality-specific importance maps are applied (for instance,

SHAP for gradient boosting, Grad-CAM/integrated gradients for convolutional networks, and attention pattern analysis for Transformers). The meta-classifier remains architecturally simple and transparently interpretable; furthermore, the local LLM is employed solely for the aggregation and verbalization of existing explanations, rather than to independently “invent” classification rationales. This approach significantly influences the formation of context for the LLM through the use of deterministic outputs from the fundamental analysis models. The described methodology aligns with research [68] concerning the reduction of hallucinations in LLMs by relying upon external sources of factual information.

Despite these inherent advantages, the chosen architecture possesses fundamental limitations. These arise from the fact that the interaction between modalities is modeled solely at the level of the base model decisions and their subsequent meta-aggregation. Even with a trainable meta-classifier capable of differentially weighting modality contributions on an instance-wise basis, the system does not construct explicit joint representations; consequently, the structure of an HTML document, the visual arrangement of elements within a screenshot, and the domain features of a URL are not considered as a singular, cross-modal attack pattern. A review [69] of deep multimodal fusion highlights that it is specifically early and hierarchical binding, employing cross-modal attention mechanisms, that facilitates the modeling of such subtle dependencies, whereas late fusion is fundamentally constrained in this specific aspect.

This implies that the system may not fully exploit potentially informative feature combinations across disparate modalities. For instance, a suspicious login form within a Document Object Model (DOM) structure, visually integrated into the corporate template of a prominent brand, and the concurrent use of a rare domain are each modeled by separate branches; they only “interact” indirectly within the output probability space of the base models. Thus, interaction remains restricted to the decision layer of the meta-classifier, whereas certain complex cross-modal patterns may only be discernible through a collective analysis of the original features [70].

The second limitation is related to the employment of a global fusion scheme. While the meta-classifier implements the weighting of contributions from the analytical branches, the base models themselves are trained independently and do not inherently adapt to the typical roles played by specific modalities within targeted phishing attack scenarios. It has been observed that a mixture-of-experts fusion scheme, or a hybrid fusion architecture incorporating cross-modal attention, facilitates the more flexible redistribution of responsibilities between modalities and the superior consideration of contextual factors—for instance, placing greater reliance upon the visual analysis branch for visually rich pages and assigning more weight to the HTML code and URL analysis for minimalist designs.

Late fusion based on stacking demonstrates the optimal balance between predictive accuracy and engineering complexity for heterogeneous data, such as URLs, HTML, and visual features. Within the proposed system, this principle is realized through the M5 meta-classifier, which formalizes late fusion as a discrete, trainable decision layer. This implementation accounts for the F1-score improvements observed in experiments I–III while preserving architectural modularity and maintaining the potential for future expansion into a hybrid model fusion framework.

The efficacy of the trainable meta-classifier is substantiated by the findings of the three computational experiments comparing unimodal models with their multimodal configurations. In computational experiment I, the meta-classifier capitalized on the respective strengths of both models: the HTML model more effectively identified attack structures, such as input forms and suspicious scripts, whereas the CatBoost model, utilizing URL features, demonstrated greater robustness against code variability and missing content.

The meta-classifier evaluated their respective contributions based on a combination of their confidence levels.

In computational experiment II, conducted on the MTLP (D2) dataset, the increase was achieved without necessitating modifications to the base model architectures; instead, improvements were achieved solely through the addition of the superior decision-making layer.

In the final experiment (experiment III), the performance of the comprehensive multimodal circuit featuring the meta-classifier was evaluated across two datasets. The resulting F1-scores were comparable to those achieved by contemporary multimodal systems designed for phishing detection that utilize late or hybrid modality fusion. Furthermore, the architecture remains modular and scalable; individual branches may be developed and retrained independently, while the meta-classifier requires only minor recalibration on a small validation subset.

Finally, the current XAI architecture primarily provides modality-specific explanations rather than elucidating shared cross-modal patterns. An SOC analyst receives discrete saliency maps for the URL, HTML, and image components, but the system lacks a built-in mechanism to demonstrate that the combination of a specific DOM element and its corresponding visual region was decisive. This level of explanatory depth requires not only late binding but also at least partial feature-level integration, an area currently being actively investigated in contemporary hybrid multimodal architectures [71].

These limitations define a logical direction for further research, which is envisioned as the subsequent phase of the system's development. The plan is to transition to a hybrid binding scheme that will retain the benefits of modularity and performance inherent in late binding, while simultaneously introducing shared latent representations and cross-modal attention mechanisms for specific modality pairings—primarily “HTML + screenshot”.

Within the explanation subsystem, the development of a hybrid architecture will facilitate the more seamless integration of XAI with multimodal representations. This approach will enable the construction of joint interpretations for cross-modal patterns, rather than being restricted to modality-specific saliency maps; moreover, it will stabilize LLM-generated explanations by grounding them more firmly in the structure of the empirical data, thus further mitigating the risk of hallucinations during the generation of analytical commentary.

Using Transformer models increases the F1-score by 5% compared to the CatBoost model, but the URL processing time, even in batch mode, drops by more than 15 times. When a certain threshold value of events per unit of time is reached, this becomes unacceptable for real-time data analysis in the SOC. Using local LLMs with prompt and zero-shot learning enables the supplementation of the results of malicious link classification with a text explanation. This increases the awareness of first- and second-line SOC monitoring specialists during both routine monitoring and incident investigation.

The multimodal approach offers several key benefits.

- Enhanced detection accuracy: It improves the detection accuracy for phishing attacks that are not identifiable by a single modality. This is because attackers face a greater challenge in bypassing all analysis modalities simultaneously.
- Reduced false positives: Using a set of models reduces the number of false positives.
- Zero-day attack detection: The system is capable of detecting novel (zero-day) attacks by analyzing a comprehensive combination of features across different modalities.
- Improved interpretability: The XAI subsystem provides insights into each modality's contribution, helping analysts to understand the rationale behind a detection decision.

The proposed architecture addresses the three primary challenges identified in the Introduction and in Section 2.7. First, robustness against attacks is enhanced through

multimodal late fusion: an adversary must simultaneously evade models based on URL features, URL symbolic analysis, HTML structure, and page appearance, while the XAI subsystem identifies anomalous patterns across all modalities. Second is the latency inherent in LLM-based systems, which is addressed by decoupling the rapid detection phase (utilizing CatBoost, CNN, and Transformer models) from the more computationally intensive generation of local LLM explanations, which are invoked only for a specific subset of cases. This approach ensures that the architecture remains compliant with the SOC's operational constraints. Third, integrating multiple modalities with XAI-driven expert support improves the efficacy against zero-day threats. In a supplementary experiment, the system identified 92 of 100 previously unrecorded URLs as suspicious, including links that were not documented in VirusTotal at the time of evaluation.

5. Conclusions

The developed system for multimodal web resource analysis demonstrated high performance, achieving F1-scores comparable to the state of the art on both a publicly available dataset (MTLP dataset, F1-score 0.953) and a continuously updated proprietary dataset (F1-score 0.989). These results were achieved while maintaining a high level of readiness for detecting zero-day attacks, a significant advantage over conventional industrial systems.

A prototype of a multimodal web resource analysis system was developed using a microservice architecture. The system is designed for high efficiency and seamless integration with a security operations center, demonstrating the ability to process 500–1100 events per second (EPS) using a subset of models.

The software implementation leverages Docker containers, RESTful interfaces, and a distributed logging system. Each component, including the Telegram interface, the analysis service, and the external threat intelligence platform module, is designed as an independent service. This architectural choice enhances the system's scalability, fault tolerance, and maintainability.

Furthermore, the proposed models were optimized for CPU execution by compiling their code, which resulted in a 26% increase in computational performance. The system's stability and operational integrity were validated through testing under conditions simulating a production environment, where it demonstrated reliable performance under loads and successful integration with SOC environments, including visualization and correlation tools such as Elasticsearch and Praeco.

Multimodal phishing detection systems that integrate explainable artificial intelligence (XAI) have achieved substantial maturity in recent years, demonstrating high accuracy rates typically ranging from 98% to 99%, while simultaneously offering enhanced decision transparency. Nevertheless, the threat landscape is continually evolving, driven by emerging adversarial threats, the proliferation of AI-generated phishing content, and the industrialization of attacks via Phishing-as-a-Service (PaaS) platforms.

The future of anti-phishing defence lies in hybrid approaches that combine the speed of traditional ML for real-time screening, the accuracy of LLMs for zero-day detection, the robustness of ensemble methods against adversarial attacks, privacy federation learning for collaborative defence, and human-in-the-loop intelligence for critical decisions.

Critical success in this domain depends on achieving an appropriate balance across several competing factors: automation versus human expertise, system performance versus interpretability, and centralized protection versus decentralized privacy. The development of a resilient defence system against the continuously evolving threat of phishing requires a multidisciplinary, collaborative effort that effectively integrates insights and methodologies from machine learning, cybersecurity, human–computer interaction, and legislative regulation.

Author Contributions: Conceptualization, A.V. and A.S. (Alexey Sulavko); methodology, A.V.; software, A.V., A.S. (Alexey Sulavko) and A.M.; validation, A.V., A.K. and A.M.; formal analysis, A.K.; investigation, V.V.; resources, A.V.; data curation, A.S. (Alexey Sulavko); writing—original draft preparation, A.V.; writing—review and editing, A.S. (Alexander Samotuga); visualization, A.K.; supervision, A.S. (Alexey Sulavko); project administration, A.S. (Alexey Sulavko); funding acquisition, A.S. (Alexey Sulavko). All authors have read and agreed to the published version of the manuscript.

Funding: This research was funded by the state assignment of the Ministry of Science and Higher Education of the Russian Federation, grant no. FSGF-2023-0004.

Institutional Review Board Statement: Not applicable.

Informed Consent Statement: Not applicable.

Data Availability Statement: All data utilized in the present research are included within the article. Further inquiries should be directed to the corresponding author.

Acknowledgments: The authors express their gratitude to Frodex LLC and ICSB LLC for providing access to computing resources and for their assistance in testing the developed software. This article is based on the results of a dissertation by A.M.

Conflicts of Interest: The authors declare no conflicts of interest.

Abbreviations

The following abbreviations are used in this manuscript:

AI	Artificial Intelligence
APWG	Anti-Phishing Working Group
CNNs	Convolutional Neural Networks
DL	Deep Learning
DOM	Document Object Model
DT	Decision Trees
EBM	Explainable Boosting Machine
ELK	Elasticsearch, Logstash, Kibana
HTML	HyperText Markup Language
IS	Information Security
kNN	k-Nearest Neighbors
LIME	Local Interpretable Model-Agnostic Explanations
LLMs	Large Language Models
LR	Logistic Regression
LSTM	Long Short-Term Memory
ML	Machine Learning
MTLP	Multitype Logo Phishing
NLP	Natural Language Processing
NNs	Neural Networks
PaaS	Phishing-as-a-Service
RF	Random Forest
ROC-AUC	Area Under the ROC Curve
SHAP	Shapley Additive Explanations
SIEM	Security Information and Event Management
SOC	Security Operations Center
SVMs	Support Vector Machines
TI	Threat Intelligence
TLD	Top-Level Domain
URL	Uniform Resource Locator
VGG	Visual Geometry Group
XAI	Explainable Artificial Intelligence

Appendix A. Excerpt from a Technical Report on the Results of the Classification of a Synthetic Web Resource, Which Was Generated Utilizing a Localized Large Language Model

Table A1. Technical appendix.

TECHNICAL APPENDIX

Complete Model Predictions

CatBoost (URL features): 98.7% phishing
 Character-CNN (URL chars): 96.2% phishing
 CodeBERT (HTML structure): 95.8% phishing
 EfficientNet-B7 (Visual): 97.8% phishing

Consensus Average: 97.1% phishing
 Standard Deviation: 1.3%
 Agreement Level: VERY HIGH

Complete SHAP Values (CatBoost)

domain_age: −0.45 (3 days old → suspicious)
 tld_type: −0.38 (.tk → free, abuse-prone)
 typosquatting_score: −0.52 (paypa1 vs paypal → deliberate)
 url_length: −0.12 (slightly long)
 subdomain_count: −0.08 (1 subdomain)
 has_https: +0.15 (has HTTPS, but...) ... [additional features]

Character-CNN Attribution Top 10

Position 8–13 “paypal”: IG attribution 0.92
 Position 13 “1”: IG attribution 0.95 (critical!)
 Position 22–24 “.tk”: IG attribution 0.88
 Position 15–21 “secure”: IG attribution 0.45 ... [full attribution map available]

CodeBERT Attention Top 10 Tokens

<input name=“creditcard”>: 0.95 weight (CRITICAL)
 <input type=“password”>: 0.89 weight

 <form action=“http://...”>: 0.82
 weight (insecure) “urgent”: 0.78
 weight “suspended”: 0.78
 weight “verify”: 0.68
 weight “account”: 0.65 weight ... [full token list available]

Visual Grad-CAM Regions (Top 5)

Table A1. Cont.

Region 1: Logo area
- Position: top-left (header)
- Activation: 0.95/1.0
- Area: 2.1% of page
- Text: "PAYPA1 Security Center"
- Elements: visual_element, square_element
Region 2: Warning banner
- Position: center (main_content)
- Activation: 1.0/1.0 (maximum!)
- Area: 8.5% of page
- Text: "URGENT: Account suspended in 24 h"
- Elements: visual_element
Region 3: Form area
- Position: center (form_area)
- Activation: 0.88/1.0
- Area: 3.2% of page
- Text: "Credit Card Number", "CVV"
- Elements: input_field (multiple)
Region 4: URL bar representation
- Position: top-center (header)
- Activation: 0.85/1.0
- Text: " https://paypal-secure.tk "
Region 5: Footer
- Position: bottom (footer)
- Activation: 0.45/1.0
- Text: "Copyright 2024 PayPa1" [with typo]
- Elements: text, poor_quality
REPORT METADATA
Report Generated: 23 October 2025 18:30:51 UTC
Analysis Duration: 3.7 s
Models Used: 4 (CatBoost, CharCNN, CodeBERT, EfficientNet-B7)
XAI Methods: 6 (SHAP, Grad-CAM 1D, IntGrad, Attention, Grad-CAM 2D, Text Convert)
LLM Model: Qwen2.5-32B-Instruct
Confidence: VERY HIGH
Analyst Review: RECOMMENDED (for quality assurance)
END OF REPORT
This report was generated by an AI-powered phishing detection system combining multiple deep learning models with explainable AI techniques.
While highly accurate, human expert review is recommended for critical cases.

References

1. Golushko, A. Current Cyber Threats: Q4 2024–Q1 2025. Available online: <https://ptsecurity.com> (accessed on 2 February 2025).
2. Group, A.P.W. Phishing Activity Trends Reports— 1Q 2025. Available online: <https://www.apwg.org> (accessed on 2 February 2025).
3. zvelo. Phishing Detection in Depth: Attack Types, Detection Tools, and More. Available online: <https://www.zvelo.com> (accessed on 2 February 2025).
4. Aslam, S.; Aslam, H.; Manzoor, A.; Chen, H.; Rasool, A. AntiPhishStack: LSTM-Based Stacked Generalization Model for Optimized Phishing URL Detection. *Symmetry* **2024**, *16*, 248. [\[CrossRef\]](#)
5. Basit, A.; Zafar, M.; Liu, X.; Javed, A.R.; Jalil, Z.; Kifayat, K. A comprehensive survey of AI-enabled phishing attacks detection techniques. *Telecommun. Syst.* **2021**, *76*, 139–154. [\[CrossRef\]](#)
6. Lukmanova, K.; Kartak, V. The development of a phishing attack protection system using software-hardware implementation of machine learning methods. *Model. Optim. Inf. Technol.* **2024**, *4*, 1–8. [\[CrossRef\]](#)
7. Kavya, S.; Sumathi, D. Staying ahead of phishers: A review of recent advances and emerging methodologies in phishing detection. *Artif. Intell. Rev.* **2024**, *58*, 50. [\[CrossRef\]](#)
8. Korniyukhina, S.; Laponina, O. Exploring the Potential of Deep Learning Algorithms to Protect Against Phishing Attacks. *Int. J. Open Inf. Technol.* **2023**, *11*, 163–174.
9. Li, W.; Manickam, S.; Chong, Y.W.; Leng, W.; Nanda, P. A State-of-the-Art Review on Phishing Website Detection Techniques. *IEEE Access* **2024**, *12*, 187976–188012. [\[CrossRef\]](#)
10. Krotov, E. Applying Deep Learning Methods to Phishing Website Detection: Performance Analysis and Model Optimization. *Curr. Res.* **2025**, *256*, 60–69.
11. Alsaedi, M.; Ghaleb, F.; Saeed, F.; Ahmad, J.; Alasli, M. Cyber Threat Intelligence-Based Malicious URL Detection Model Using Ensemble Learning. *Sensors* **2022**, *22*, 3373. [\[CrossRef\]](#)
12. Aljofey, A.; Jiang, Q.; Rasool, A.; Chen, H.; Liu, W.; Qu, Q.; Wang, Y. An effective detection approach for phishing websites using URL and HTML features. *Sci. Rep.* **2022**, *12*, 8842. [\[CrossRef\]](#)
13. Wangchuk, T.; Gonsalves, T. Multimodal Phishing Detection on Social Networking Sites: A Systematic Review. *IEEE Access* **2025**, *13*, 103405–103416. [\[CrossRef\]](#)
14. Baltrušaitis, T.; Ahuja, C.; Morency, L.P. Multimodal Machine Learning: A Survey and Taxonomy. *IEEE Trans. Pattern Anal. Mach. Intell.* **2019**, *41*, 423–443. [\[CrossRef\]](#)
15. Govindaraaj, J. The Role of Explainable AI in Understanding Phishing Susceptibility. *J. Recent Trends Comput. Sci. Eng. JRTCSE* **2024**, *12*, 1–6.
16. Calzarossa, M.C.; Giudici, P.; Zieni, R. An assessment framework for explainable AI with applications to cybersecurity. *Artif. Intell. Rev.* **2025**, *58*, 150. [\[CrossRef\]](#)
17. Shyni, E. Enhancing Phishing Detection with Explainable AI (XAI): A Transparent Cybersecurity Approach. 2025. Available online: <https://smartsecuritytips.com/phishing-xai/> (accessed on 2 February 2025).
18. Vrbančič, G.; Fister, I.; Podgorelec, V. Datasets for phishing websites detection. *Data Brief* **2020**, *33*, 106438. [\[CrossRef\]](#)
19. Al-diabat, M. Detection and Prediction of Phishing Websites using Classification Mining Techniques. *Int. J. Comput. Appl.* **2016**, *147*, 5–11. [\[CrossRef\]](#)
20. Almomani, A.; Alauthman, M.; Shatnawi, M.T.; Alweshah, M.; Alrosan, A.; Alomoush, W.; Gupta, B.B.; Gupta, B.B.; Gupta, B.B. Phishing Website Detection With Semantic Features Based on Machine Learning Classifiers: A Comparative Study. *Int. J. Semant. Web Inf. Syst.* **2022**, *18*, 1–24. [\[CrossRef\]](#)
21. Alazaidah, R.; Al-Shaikh, A.; Almousa, M.; Khafajeh, H.; Samara, G.; Alzyoud, M.; Al-shanableh, N.; Almatarneh, S. Website Phishing Detection Using Machine Learning Techniques. *J. Stat. Appl. Probab.* **2024**, *13*, 119–129. [\[CrossRef\]](#)
22. Mahajan, R.; Siddavatam, I. Phishing Website Detection using Machine Learning Algorithms. *Int. J. Comput. Appl.* **2018**, *181*, 45–47. [\[CrossRef\]](#)
23. Khera, M.; Prasad, T.; Xess, L.; Singh, R.; Aiden, M.K. Malicious Website Detection using Machine Learning. *Int. J. Eng. Res. Technol. (IJERT)* **2022**, *11*, 19–27. [\[CrossRef\]](#)
24. Lee, J.; Lim, P.; Hooi, B.; Divakaran, D.M. Multimodal Large Language Models for Phishing Webpage Detection and Identification. In Proceedings of the 2024 APWG Symposium on Electronic Crime Research (eCrime), Boston, MA, USA, 24–26 September 2024; pp. 1–13, Version Number: 1. [\[CrossRef\]](#)
25. Syed, S.A. AI-Driven Detection of Phishing Attacks through Multimodal Analysis of Content and Design. *Int. J. Innov. Res. Comput. Commun. Eng.* **2024**, *12*, 9979–9987. [\[CrossRef\]](#)
26. Trad, F.; Chehab, A. Large Multimodal Agents for Accurate Phishing Detection with Enhanced Token Optimization and Cost Reduction. In Proceedings of the 2024 2nd International Conference on Foundation and Large Language Models (FLLM), Dubai, United Arab Emirates, 26–29 November 2024; pp. 229–237. [\[CrossRef\]](#)

27. Cao, T.; Huang, C.; Li, Y.; Wang, H.; He, A.; Oo, N.; Hooi, B. PhishAgent: A Robust Multimodal Agent for Phishing Webpage Detection. In Proceedings of the Thirty-Ninth AAAI Conference on Artificial Intelligence and Thirty-Seventh Conference on Innovative Applications of Artificial Intelligence and Fifteenth Symposium on Educational Advances in Artificial Intelligence (AAAI'25/IAAI'25/EAAI'25), Philadelphia, PA, USA, 25 February–4 March 2025; VAAAI Press: Washington, DC, USA, 2025; Volume 39, pp. 27869–27877. [\[CrossRef\]](#)
28. Li, Y.; Huang, C.; Deng, S.; Lock, M.L.; Cao, T.; Oo, N.; Lim, H.W.; Hooi, B. KnowPhish: Large Language Models Meet Multimodal Knowledge Graphs for Enhancing Reference-Based Phishing Detection. In Proceedings of the 33rd USENIX Conference on Security Symposium (SEC '24) USENIX Association, Philadelphia, PA, USA, 14–16 August 2024; pp. 793–810. [\[CrossRef\]](#)
29. Alshingiti, Z.; Alaql, R.; Al-Muhtadi, J.; Haq, Q.E.U.; Saleem, K.; Faheem, M.H. A Deep Learning-Based Phishing Detection System Using CNN, LSTM, and LSTM-CNN. *Electronics* **2023**, *12*, 232. [\[CrossRef\]](#)
30. Aldakheel, E.A.; Zakariah, M.; Gashgari, G.A.; Almarshad, F.A.; Alzahrani, A.I.A. A Deep Learning-Based Innovative Technique for Phishing Detection in Modern Security with Uniform Resource Locators. *Sensors* **2023**, *23*, 4403. [\[CrossRef\]](#)
31. Repository, U.M.L. PhiUSIL Phishing URL Dataset. Available online: <https://archive.ics.uci.edu/dataset/967> (accessed on 2 February 2025).
32. Chiew, K.L.; Tan, C.L.; Wong, K.; Yong, K.S.; Tiong, W.K. A new hybrid ensemble feature selection framework for machine learning-based phishing detection system. *Inf. Sci.* **2019**, *484*, 153–166. [\[CrossRef\]](#)
33. Shafin, S.S. An explainable feature selection framework for web phishing detection with machine learning. *Data Sci. Manag.* **2025**, *8*, 127–136. [\[CrossRef\]](#)
34. Yoon, J.H.; Buu, S.J.; Kim, H.J. Phishing Webpage Detection via Multi-Modal Integration of HTML DOM Graphs and URL Features Based on Graph Convolutional and Transformer Networks. *Electronics* **2024**, *13*, 3344. [\[CrossRef\]](#)
35. Asiri, S.; Xiao, Y.; Li, T. PhishTransformer: A Novel Approach to Detect Phishing Attacks Using URL Collection and Transformer. *Electronics* **2023**, *13*, 30. [\[CrossRef\]](#)
36. Lin, Y.; Liu, R.; Divakaran, D.M.; Ng, J.Y.; Chan, Q.Z.; Lu, Y.; Si, Y.; Zhang, F.; Dong, J.S. Phishpedia: A Hybrid Deep Learning Based Approach to Visually Identify Phishing Webpages. In Proceedings of the 30th USENIX Security Symposium (USENIX Security 21), USENIX Association, Vancouver, BC, Canada, 11–13 August 2021; pp. 3793–3810. Available online: <https://www.usenix.org/system/files/sec21-lin.pdf> (accessed on 2 February 2025).
37. Abdelnabi, S.; Krombholz, K.; Fritz, M. VisualPhishNet: Zero-Day Phishing Website Detection by Visual Similarity. In Proceedings of the ACM Conference on Computer and Communications Security (CCS), Virtual Event USA, 9–13 November 2020; ACM: New York, NY, USA, 2020. Available online: https://publications.cispa.de/articles/conference_contribution/VisualPhishNet_Zero-Day_Phishing_Website_Detection_by_Visual_Similarity/24613248 (accessed on 2 February 2025).
38. Zhang, L.; Zhang, P.; Liu, L.; Tan, J. Multiphish: Multi-Modal Features Fusion Networks for Phishing Detection. In Proceedings of the ICASSP 2021—2021 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP), Toronto, ON, Canada, 6–11 June 2021; pp. 3520–3524. [\[CrossRef\]](#)
39. Salih, A.M.; Raisi-Estabragh, Z.; Galazzo, I.B.; Radeva, P.; Petersen, S.E.; Lekadir, K.; Menegaz, G. A Perspective on Explainable Artificial Intelligence Methods: SHAP and LIME. *Adv. Intell. Syst.* **2025**, *7*, 2400304. [\[CrossRef\]](#)
40. Lim, B.; Huerta, R.; Sotelo, A.; Quintela, A.; Kumar, P. EXPLICATE: Enhancing Phishing Detection through Explainable AI and LLM-Powered Interpretability. *arXiv* **2025**, arXiv:2503.20796. [\[CrossRef\]](#)
41. Shendkar, B.D.; Chandre, P.R.; Madachane, S.S.; Kulkarni, N.; Deshmukh, S. Enhancing Phishing Attack Detection Using Explainable AI: Trends and Innovations. *ASEAN J. Sci. Technol. Dev.* **2024**, *42*, 8. [\[CrossRef\]](#)
42. Galego Hernandez, P.R.; Floret, C.P.; Cardozo De Almeida, K.F.; Da Silva, V.C.; Papa, J.P.; Pontara Da Costa, K.A. Phishing Detection Using URL-based XAI Techniques. In Proceedings of the 2021 IEEE Symposium Series on Computational Intelligence (SSCI), Orlando, FL, USA, 4–7 December 2021; pp. 01–06. [\[CrossRef\]](#)
43. Akhtar, H.M.U.; Nauman, M.; Akhtar, N.; Hameed, M.; Hameed, S.; Tareen, M.Z. Mitigating Cyber Threats: Machine Learning and Explainable AI for Phishing Detection. *VFAST Trans. Softw. Eng.* **2025**, *13*, 170–195. [\[CrossRef\]](#)
44. Rashid, F.; Ranaweera, N.; Doyle, B.; Seneviratne, S. LLMs are One-Shot URL Classifiers and Explainers. *Comput. Netw.* **2024**, *258*, 170–195. [\[CrossRef\]](#)
45. Feng, Z.; Guo, D.; Tang, D.; Duan, N.; Feng, X.; Gong, M.; Shou, L.; Qin, B.; Liu, T.; Jiang, D.; et al. CodeBERT: A Pre-Trained Model for Programming and Natural Languages. In Proceedings of the Findings of the Association for Computational Linguistics: EMNLP 2020, Online, 16–20 November 2020; pp. 1536–1547. [\[CrossRef\]](#)
46. Moreau, L. AI Explainability with Grad-CAM: Visualizing Neural Network Decisions. 2025. Available online: <https://www.edgeimpulse.com/blog/ai-explainability-with-grad-cam-visualizing-neural-network-decisions/> (accessed on 2 February 2025).
47. Shrikumar, A.; Greenside, P.; Kundaje, A. Learning Important Features Through Propagating Activation Differences. In Proceedings of the 34th International Conference on Machine Learning—Volume 70 (ICML'17), JMLR.org, Sydney, Australia, 6–11 August 2017; pp. 3145–3153. Available online: <https://dl.acm.org/doi/10.5555/3305890.3306006> (accessed on 19 December 2025).

48. Nori, H.; Jenkins, S.; Koch, P.; Caruana, R. InterpretML: A Unified Framework for Machine Learning Interpretability. *arXiv* **2019**, arXiv:1909.09223. Available online: <https://api.semanticscholar.org/CorpusID:202712518> (accessed on 19 December 2025).
49. Colhak, F.; Ecevit, M.I.; Dag, H. Transfer Learning for Phishing Detection: Screenshot-Based Website Classification. In Proceedings of the 2024 9th International Conference on Computer Science and Engineering (UBMK), Antalya, Turkiye, 26–28 October 2024; pp. 1–6. [CrossRef]
50. Datasets, H. Combined BERT Phishing Dataset. Available online: <https://huggingface.co/datasets/ealvaradob/phishing-dataset> (accessed on 2 February 2025).
51. Alexa Top 1 Million Sites. Available online: <https://www.kaggle.com/datasets/cheedcheed/top1m> (accessed on 14 November 2025).
52. Mendoza Vega, D.O.; Diaz Mercado, D.O.; Cardenas, E.J.E. PhishFind: A Machine Learning-Based System for Real-Time Phishing Detection. In Proceedings of the 2025 8th International Conference on Systems Engineering—Cybersecurity & AI: Building a Reliable Digital Future (CIIS '25), Lima, Peru, 1–3 October 2025; Association for Computing Machinery: New York, NY, USA, 2025; pp. 31–39. [CrossRef]
53. Yuan, Y.; Hao, Q.; Apruzzese, G.; Conti, M.; Wang, G. “Are Adversarial Phishing Webpages a Threat in Reality?” Understanding the Users’ Perception of Adversarial Webpages. In Proceedings of the ACM Web Conference 2024, Singapore, 13–17 May 2024; pp. 1712–1723. [CrossRef]
54. Akiba, T.; Sano, S.; Yanase, T.; Ohta, T.; Koyama, M. Optuna: A Next-generation Hyperparameter Optimization Framework. In Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining, Anchorage, AK, USA, 4–8 August 2019; pp. 2623–2631. [CrossRef]
55. Prokhorenkova, L.; Gusev, G.; Vorobev, A.; Dorogush, A.V.; Gulin, A. CatBoost: Unbiased boosting with categorical features. In *Advances in Neural Information Processing Systems*; Curran Associates, Inc.: Red Hook, NY, USA, 2018; Volume 31.
56. Tan, M.; Le, Q.V. EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks. In Proceedings of the 36th International Conference on Machine Learning; ICML 2019, Long Beach, CA, USA, 9–15 June 2019; pp. 6105–6114. Available online: <https://proceedings.mlr.press/v97/tan19a.html> (accessed on 19 December 2025).
57. Tekkali, C.G.; Natarajan, K. Transfer learning of pre-trained CNNs on digital transaction fraud detection. *Int. J. Knowl.-Based Intell. Eng. Syst.* **2024**, *28*, 571–580. [CrossRef]
58. Cik, I.; Rasamoelina, A.D.; Mach, M.; Sincak, P. Explaining Deep Neural Network using Layer-wise Relevance Propagation and Integrated Gradients. In Proceedings of the 2021 IEEE 19th World Symposium on Applied Machine Intelligence and Informatics (SAMII), Herl’any, Slovakia, 21–23 January 2021; pp. 000381–000386. [CrossRef]
59. Ali, A.; Schnake, T.; Eberle, O.; Montavon, G.; Müller, K.R.; Wolf, L. XAI for Transformers: Better Explanations through Conservative Propagation, 2022. In Proceedings of the 39th International Conference on Machine Learning, Baltimore, MD, USA, 17–23 July 2022; PMLR 162, pp. 1–17. Available online: <https://proceedings.mlr.press/v162/ali22a/ali22a.pdf> (accessed on 19 December 2025).
60. Yang, A.; Yang, B.; Hui, B.; Zheng, B.; Yu, B.; Zhou, C.; Li, C.; Li, C.; Liu, D.; Huang, F.; et al. Qwen2 Technical Report. *arXiv* **2024**, arXiv:2407.10671. . [CrossRef]
61. Colhak, F.; Ecevit, M.I.; Ucar, B.E.; Creutzburg, R.; Dag, H. Phishing Website Detection through Multi-Model Analysis of HTML Content. In *Proceedings of International Conference on Theoretical and Applied Computing*; Springer: Singapore, 2025; pp. 171–184. [CrossRef]
62. Younis, E.M.G.; Zaki, S.M.; Kanjo, E.; Houssein, E.H. Evaluating Ensemble Learning Methods for Multi-Modal Emotion Recognition Using Sensor Data Fusion. *Sensors* **2022**, *22*, 5611. [CrossRef] [PubMed]
63. Lutskovich, A.; Vasilyev, V.; Vulfin, A.; Kirillova, A.; Sulavko, A. Automated system for analyzing weakly structured threat intelligence data using large language models. *Informatsionno-Upr. Sist. Inf. Control Syst.* **2025**, *2*, 50–67. [CrossRef]
64. Caruso, C.M.; Soda, P.; Guarrasi, V. MARIA: A multimodal transformer model for incomplete healthcare data. *Comput. Biol. Med.* **2025**, *196*, 110843. [CrossRef] [PubMed]
65. Rahimi, F.; Tamantini, C.; Orlandini, A.; Fracasso, F.; Siciliano, R. Comparing Fusion Strategies for Multimodal Emotion Prediction Using Deep Physiological Features. In Proceedings of the Workshop on Social Robotics for Human-Centered Assistive and Rehabilitation Co-Located with International Conference on Social Robotics + AI (ICSR + AI 2025). CEUR-WS, Naples, Italy, 10–12 September 2025; pp. 56–64. Available online: <https://ceur-ws.org/Vol-4067/paper9.pdf> (accessed on 28 December 2025).
66. Liang, Z.; Qu, A.; Shahbaba, B. Meta Fusion: A Unified Framework For Multimodality Fusion with Mutual Learning. *arXiv* **2025**, arXiv:2507.20089. [CrossRef]
67. Stahlschmidt, S.R.; Ulfenborg, B.; Synnergren, J. Multimodal deep learning for biomedical data fusion: A review. *Briefings Bioinform.* **2022**, *23*, bbab569. [CrossRef]
68. Kalai, A.T.; Nachum, O.; Vempala, S.S.; Zhang, E. Why Language Models Hallucinate. *arXiv* **2025**, arXiv:2509.04664. [CrossRef]
69. Zhao, F.; Zhang, C.; Geng, B. Deep Multimodal Data Fusion. *ACM Comput. Surv.* **2024**, *56*, 216. [CrossRef]

70. Belfedhal, A.E. Multi-Modal Deep Learning for Effective Malicious Webpage Detection. *Rev. D'Intell. Artif.* **2023**, *37*, 1005–1013. [[CrossRef](#)]
71. Wang, C.; Zhang, M.; Shi, F.; Xue, P.; Li, Y. A Hybrid Multimodal Data Fusion-Based Method for Identifying Gambling Websites. *Electronics* **2022**, *11*, 2489. [[CrossRef](#)]

Disclaimer/Publisher's Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.